

CONVOLUTIONAL NEURAL NETWORK

Nimisha Roy
Georgia Tech

Great visualization tool:
<https://poloclub.github.io/cnn-explainer/>

Facial Recognition

Deep Learning Neural Networks use layers of increasingly complex rules to categorize Complicated shapes such as faces

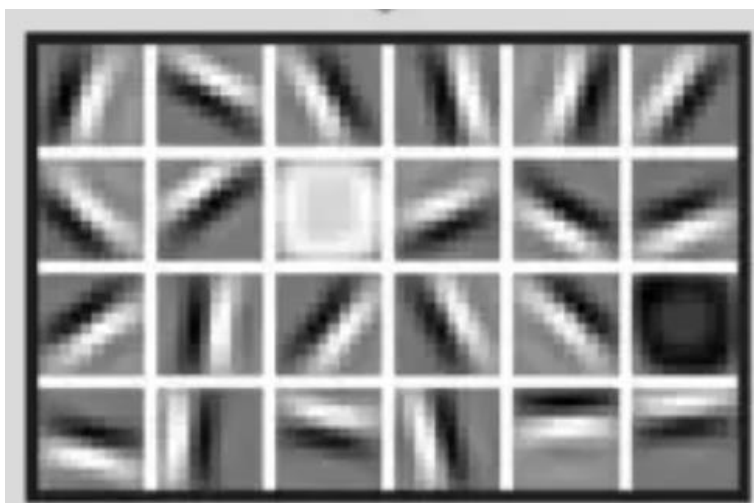
Layer 1: The model identifies pixels of light and dark



Layer 3: The model learns to identify more complex shapes and objects



Layer 2: The model learns to identify edges and simple shapes



Layer 4: The model learns which shapes and objects can be used to define a human face

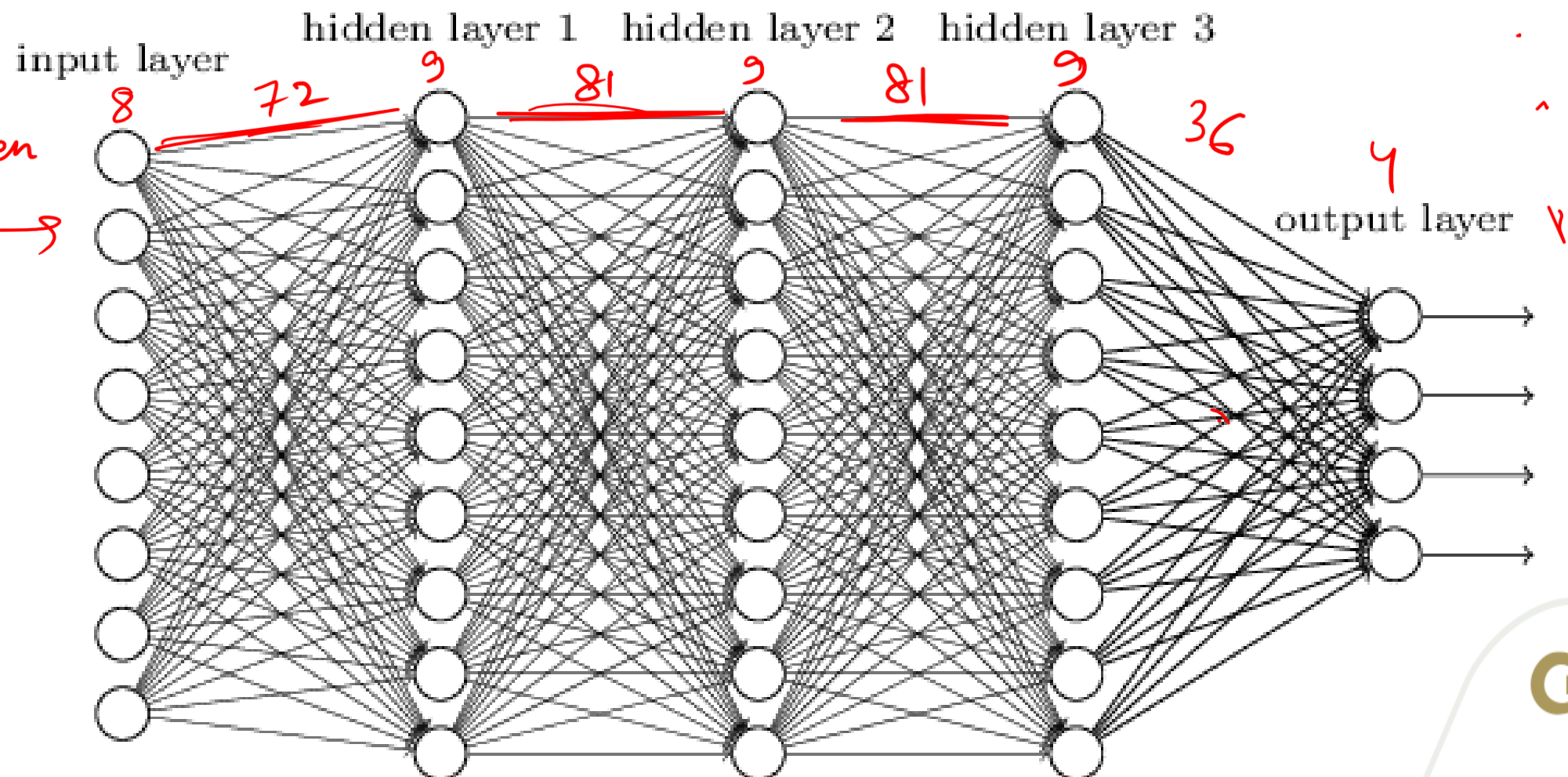


Smaller Network: CNN

- We know it is good to learn a small model.
- From this fully connected model, do we really need all the edges?
- Can some of these be shared?

parameters can be
drastically reduced by
SHARING

flatten
1000's



Weight
that CNN
needs to learn *

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Average / Blur

$$\text{filter}_{\text{flat}} = \left[\frac{1}{9} \ \frac{1}{9} \ \frac{1}{9} \ \dots \ \frac{1}{9} \right]_{1 \times 9}$$

$$\text{Subimage}_{\text{flat}} = [1 \ 3 \ 2 \ 0 \ 1 \ 0 \ 0 \ 7 \ 1]_{1 \times 9}$$

grayscale

1	3	2	6	0	1
0	1	0	0	1	0
0	7	1	1	0	0
1	0	4	5	1	8
4	1	0	7	2	0
3	0	1	0	1	0

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

feature
map

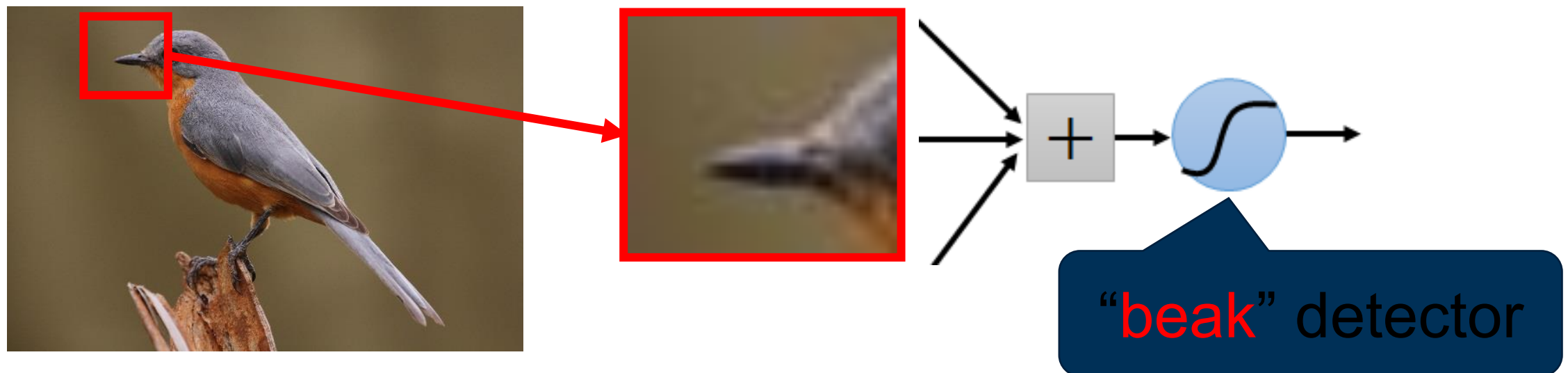


Consider learning an image:

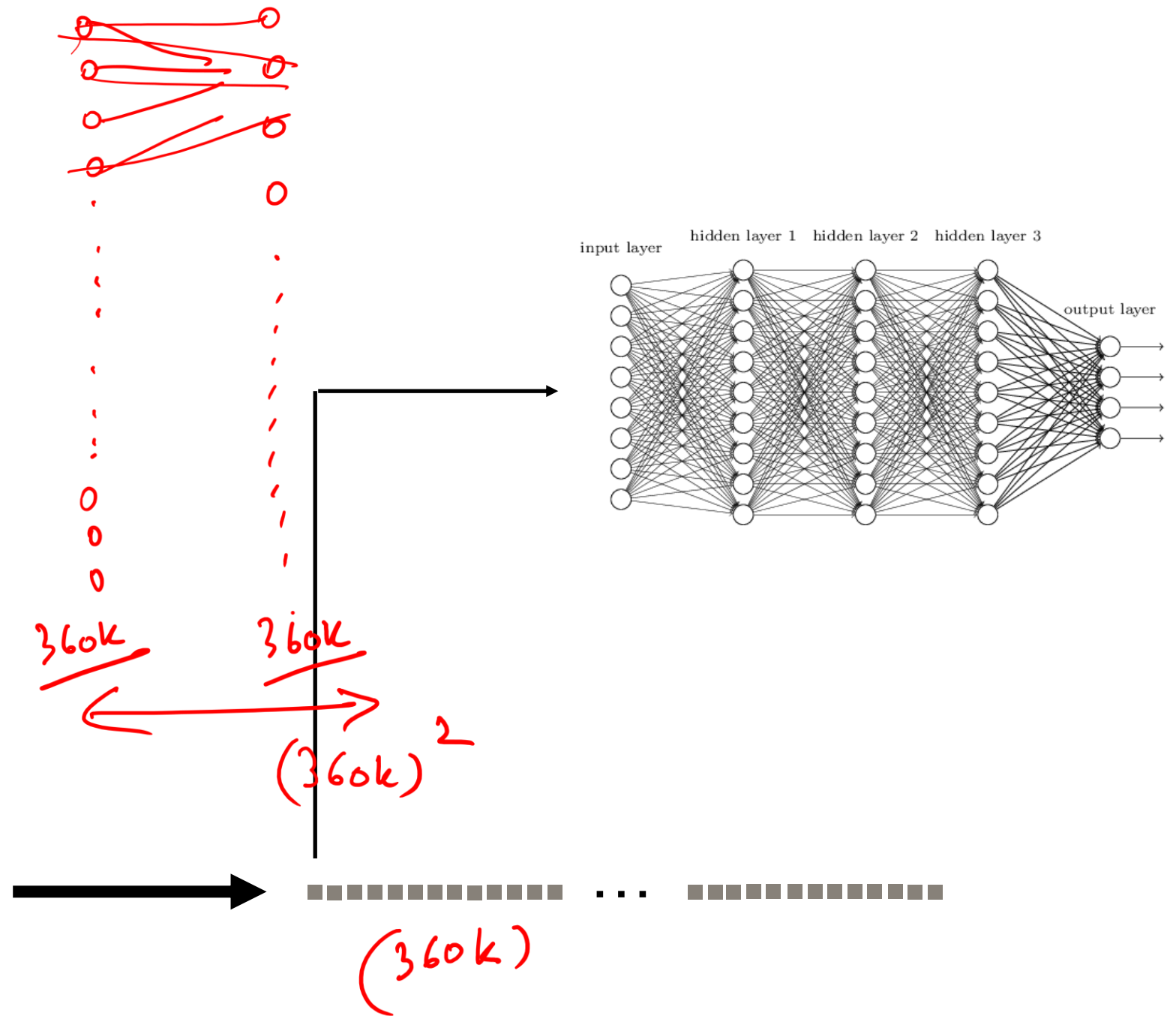
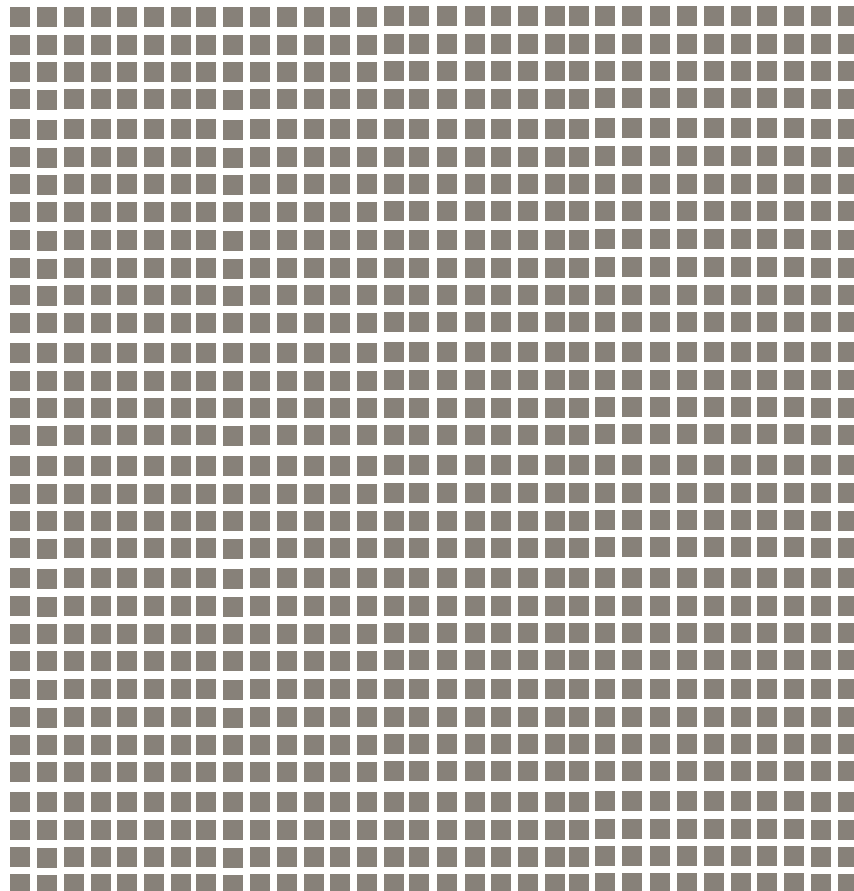
Beak Detection

- Some patterns are much smaller than the whole image

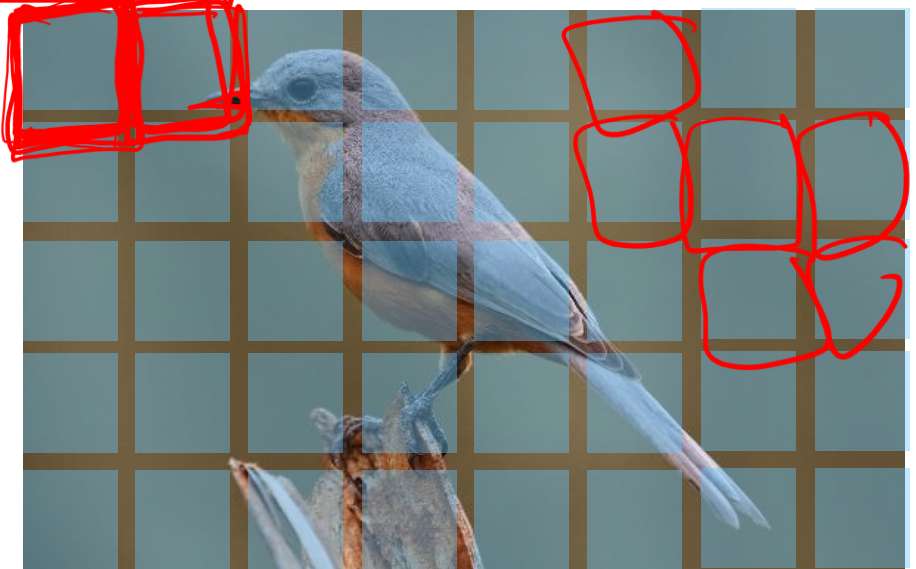
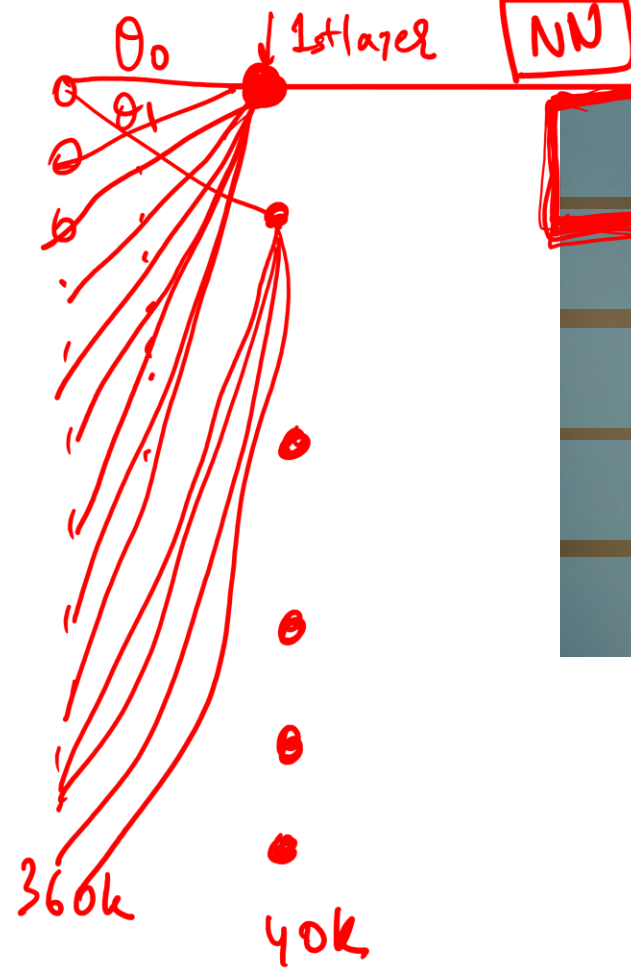
Can represent a small region with fewer parameters



600x600

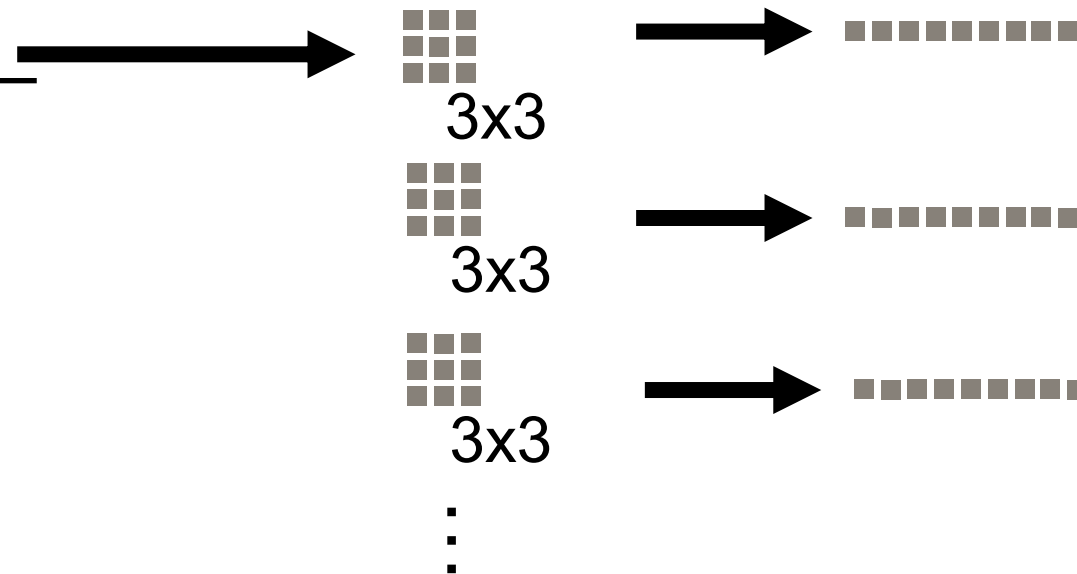
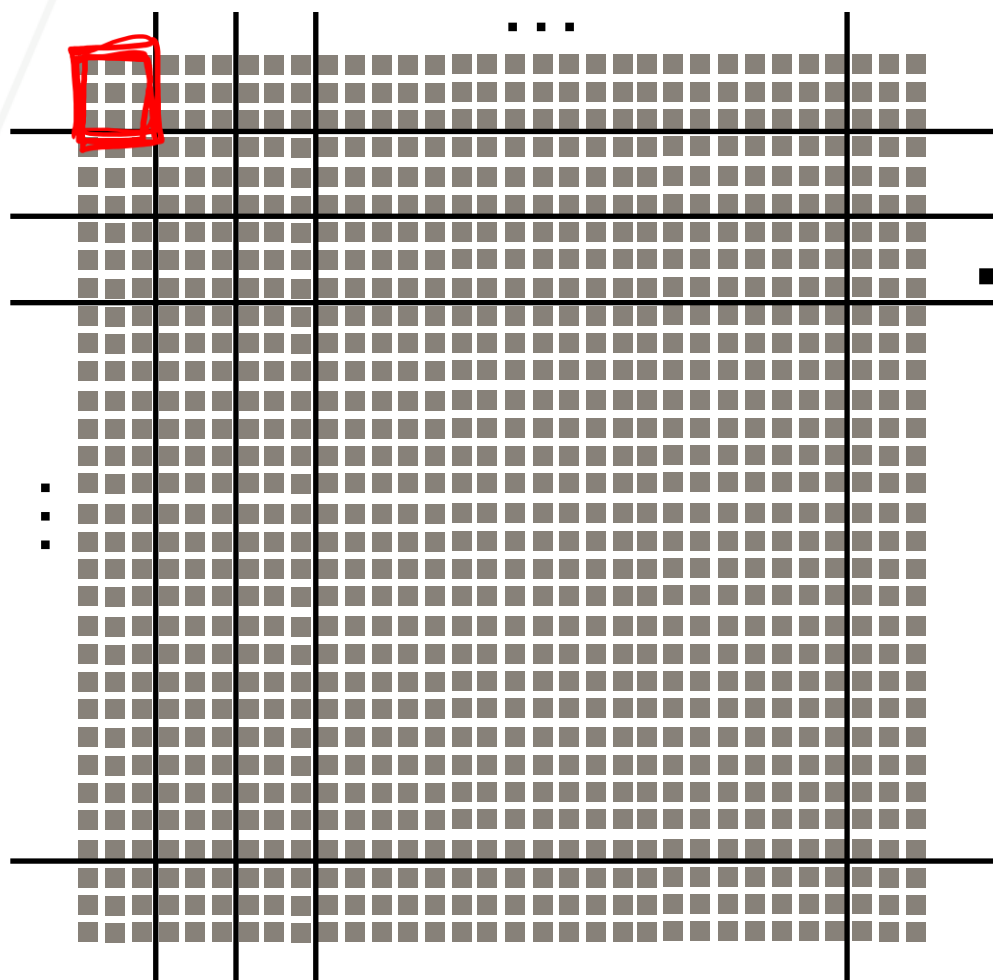


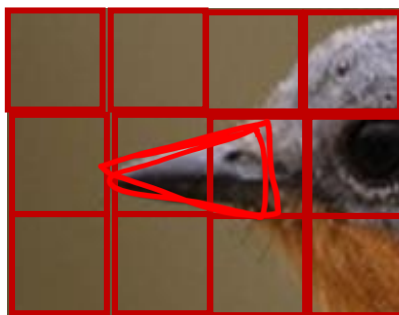
600x600



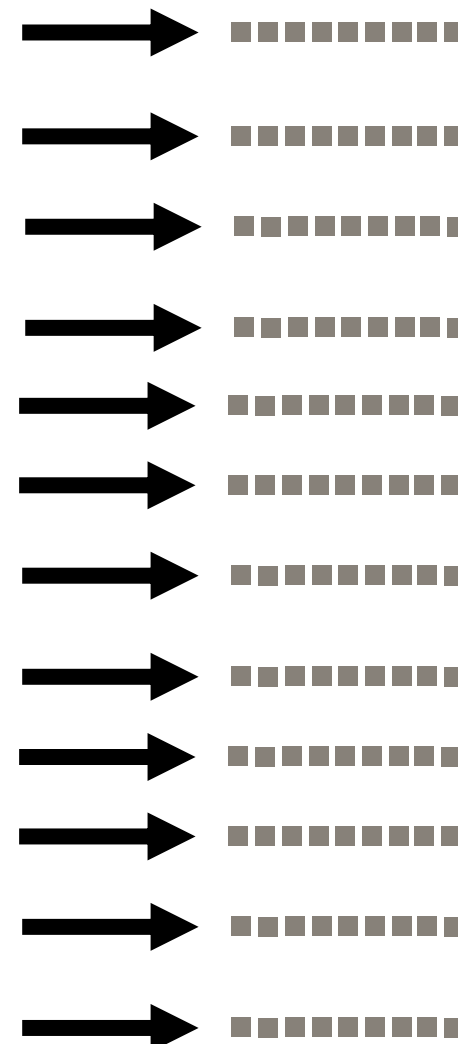
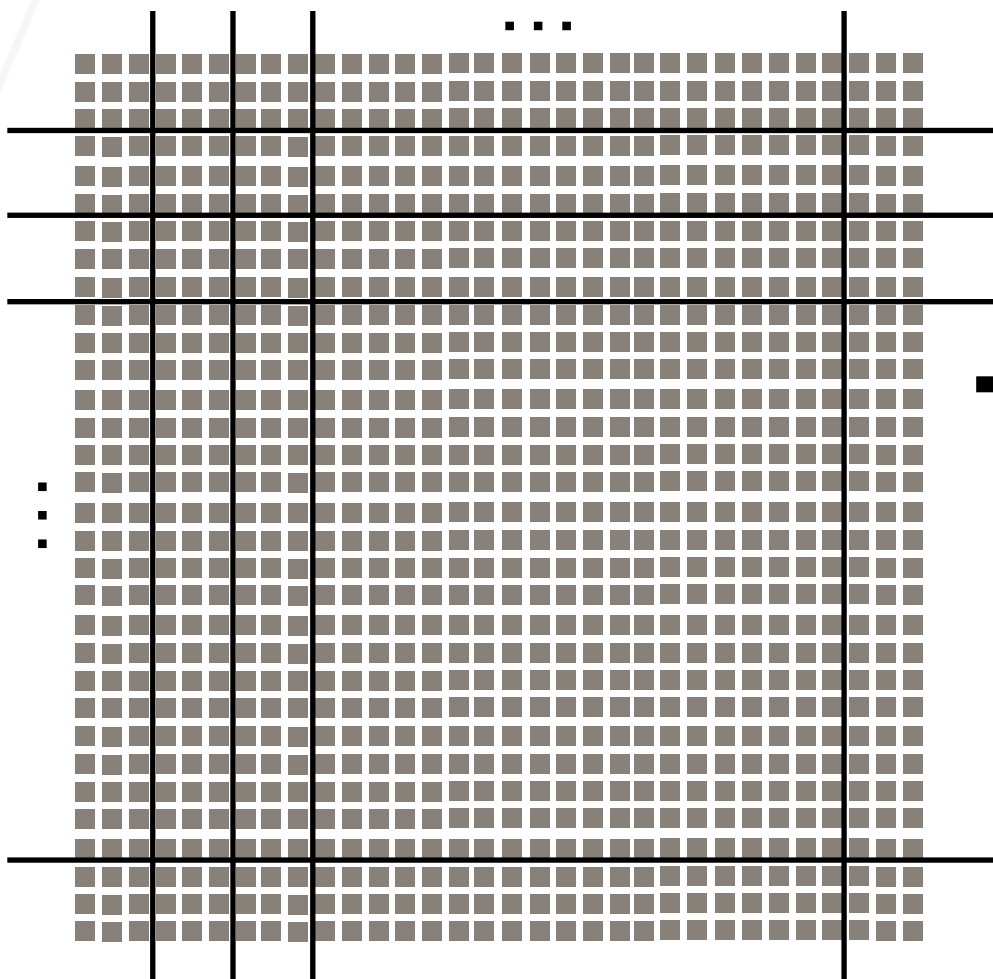
Handwritten annotations for the grid:

- $\theta_0 \dots \theta_8$ — LOW
- $\theta_9 \dots \theta_{16}$ — HIGH





9x12



...

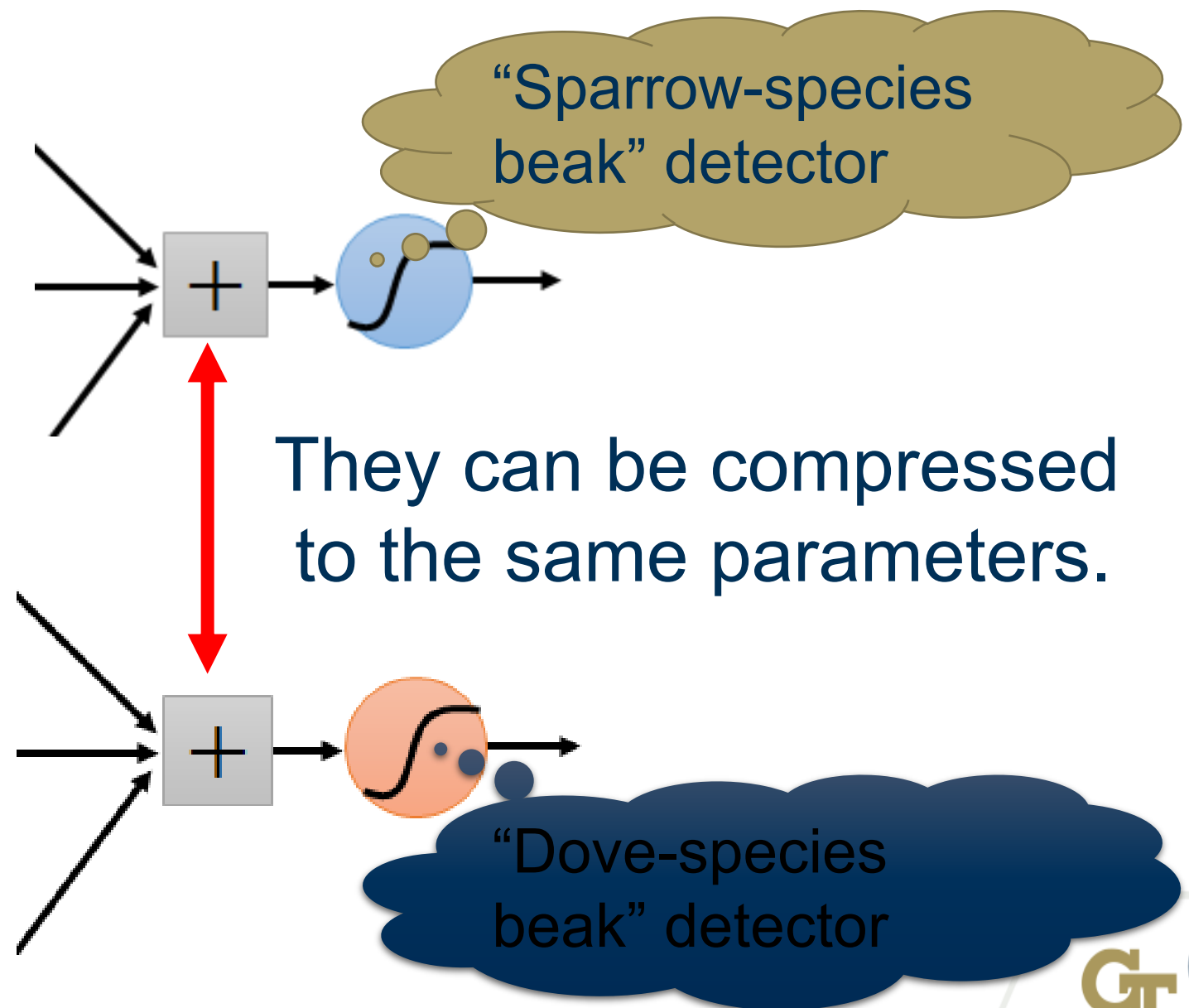
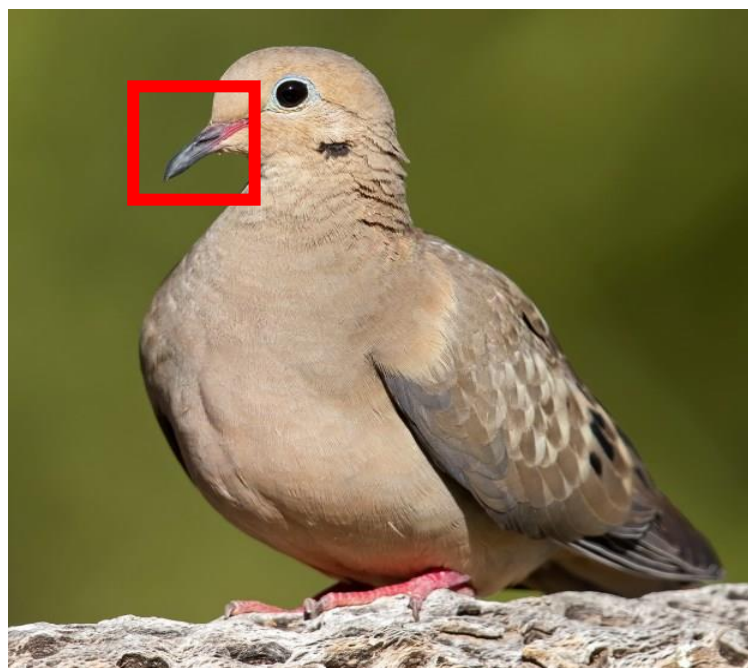
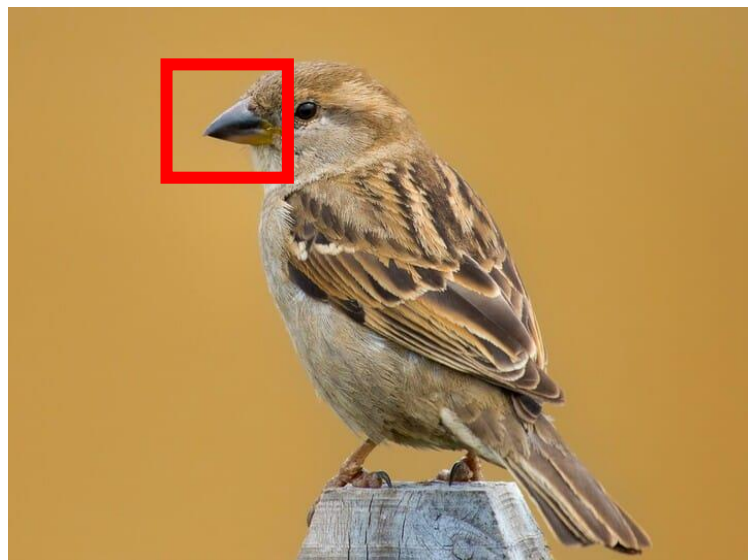
...

Why is NN wasteful?

- Each neuron in a fully connected network “sees” *every pixel*, no matter where it is in the image.
But that’s unnecessary for images — because important features like **beaks**, **wings**, **eyes** tend to be **local** and **repeat** in different places.
- So, in a standard NN, you’d need to *independently learn a beak detector* for:
 - every position (top-left, center, bottom-right),
 - every bird,
 - every image.
- Each location would require **its own set of weights**, hence *millions* of parameters.

Same pattern appears in different places:
They can be compressed!

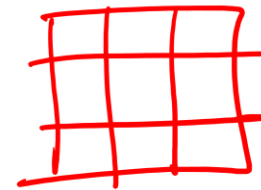
**What about training a lot of such “small” detectors
and each detector must “move around”.**



How CNNs fix this with parameter sharing

CNNs are an advanced version of NNs that use specialized layers with filters to automatically and efficiently learn features like edges, shapes, and objects from data.

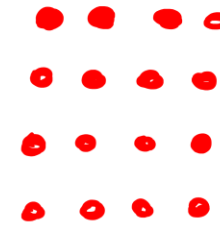
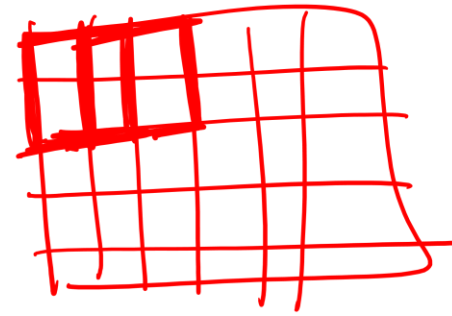
“Instead of learning a separate detector for every position, let’s **use one small filter** (like a 3×3 beak pattern) and **slide it** over the whole image.”



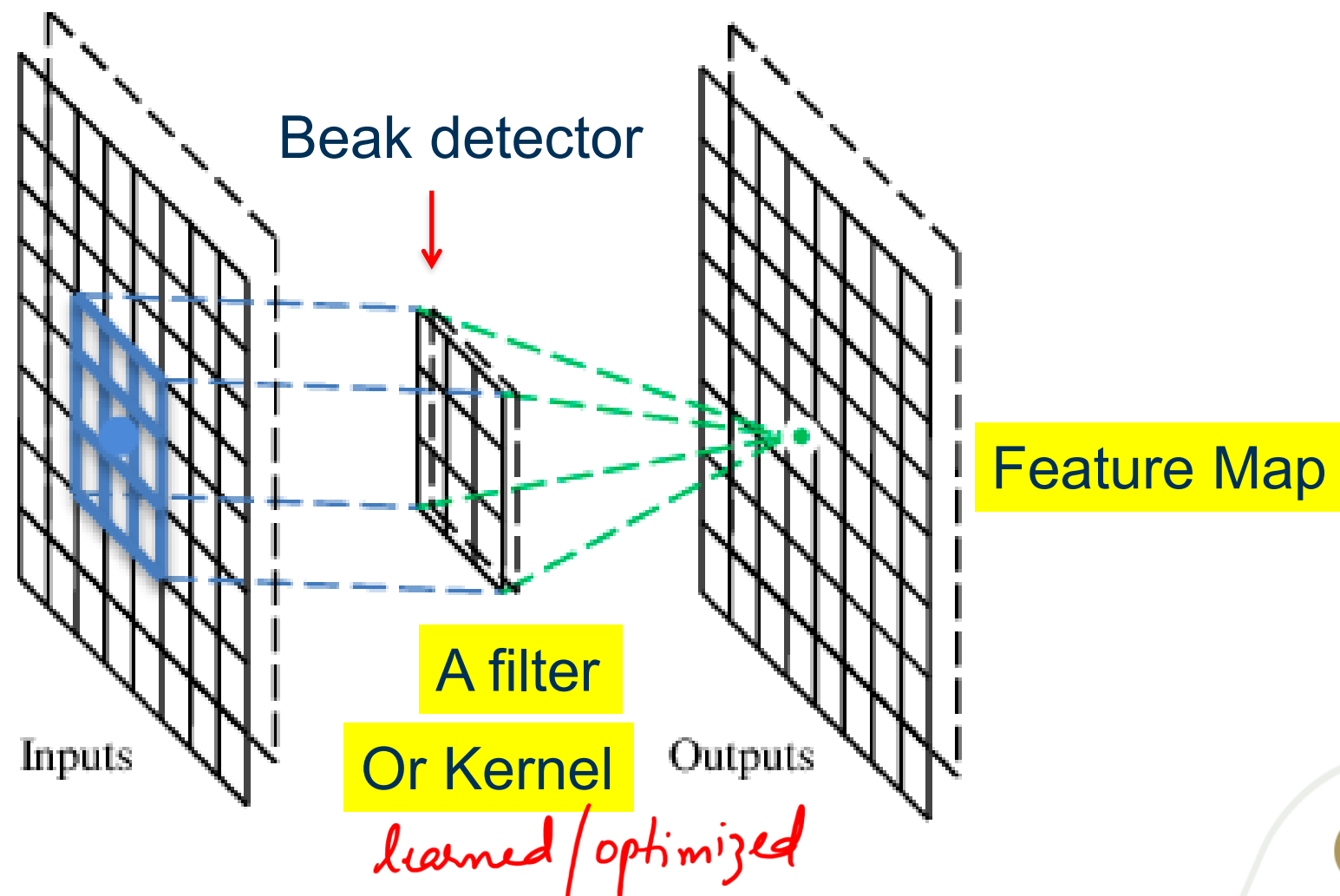
Now:

- You only need **9 weights** (for a 3×3 filter),
- And you reuse them across *all locations*.
- That reuse — the **same weights everywhere** — is **parameter sharing**.

A convolutional layer



A CNN is a neural network with some convolutional layers (and some other layers). A convolutional layer has a number of filters that does convolutional operation.



Convolution

We want to detect vertical lines and diagonal lines in the binary image

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

learned by CNN
These are the network parameters to be learned.

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

Convolution

$$\text{fil}_{\text{flat}} = [1 \ -1 \ -1 \ -1 \ 1 \ 1 \ -1 \ 1 \ 1]$$
$$\text{Sub}_{\text{flat}} = [1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1]$$

stride=1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot
product



6 x 6 image

Convolution

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3

-3

Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

4x4 feature map

Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

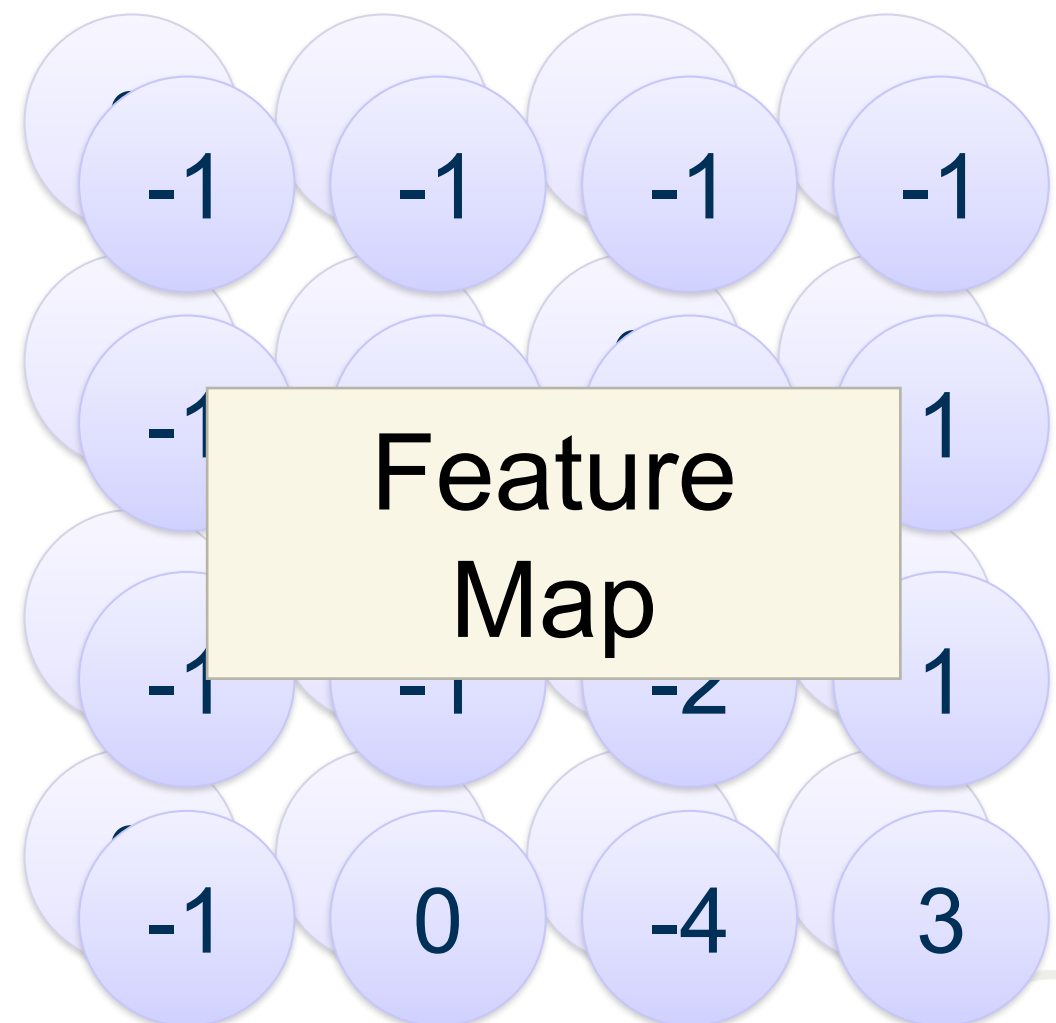
6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

optimized

Repeat this for each filter



Two 4 x 4 images
Forming 2 x 4 x 4 matrix

How Convolution Learns to Detect Features

① Local Feature Detection

- Each **filter** in a CNN acts as a *small pattern detector*.
- A filter that “looks like” a **beak shape** will produce **high activation** wherever a similar pattern appears in the image.
- In the feature map, these high activations mark **where the beak is located**.

② Parameter Sharing Across Locations

- The same learned filter slides across the image — this means the **same weights** detect the same pattern *anywhere*.
- A filter trained to recognize a **sparrow’s beak** will also respond to a **dove’s beak** if the shape is similar.
- This ability to reuse the same detector across different positions is what makes CNNs **efficient and translation-invariant**.
-

• ③ Multiple Filters, Multiple Features

- CNNs learn **many filters**, each detecting a different feature (edges, textures, beaks, wings, eyes, etc.).
- These feature maps stack together, allowing the network to recognize **multiple parts of an object** simultaneously.
- Higher layers then combine these local detections to recognize **entire objects or categories** (e.g., “bird”).

Color image: RGB 3 channels

$$fil_{flat} = [\dots]_{1 \times 27}$$

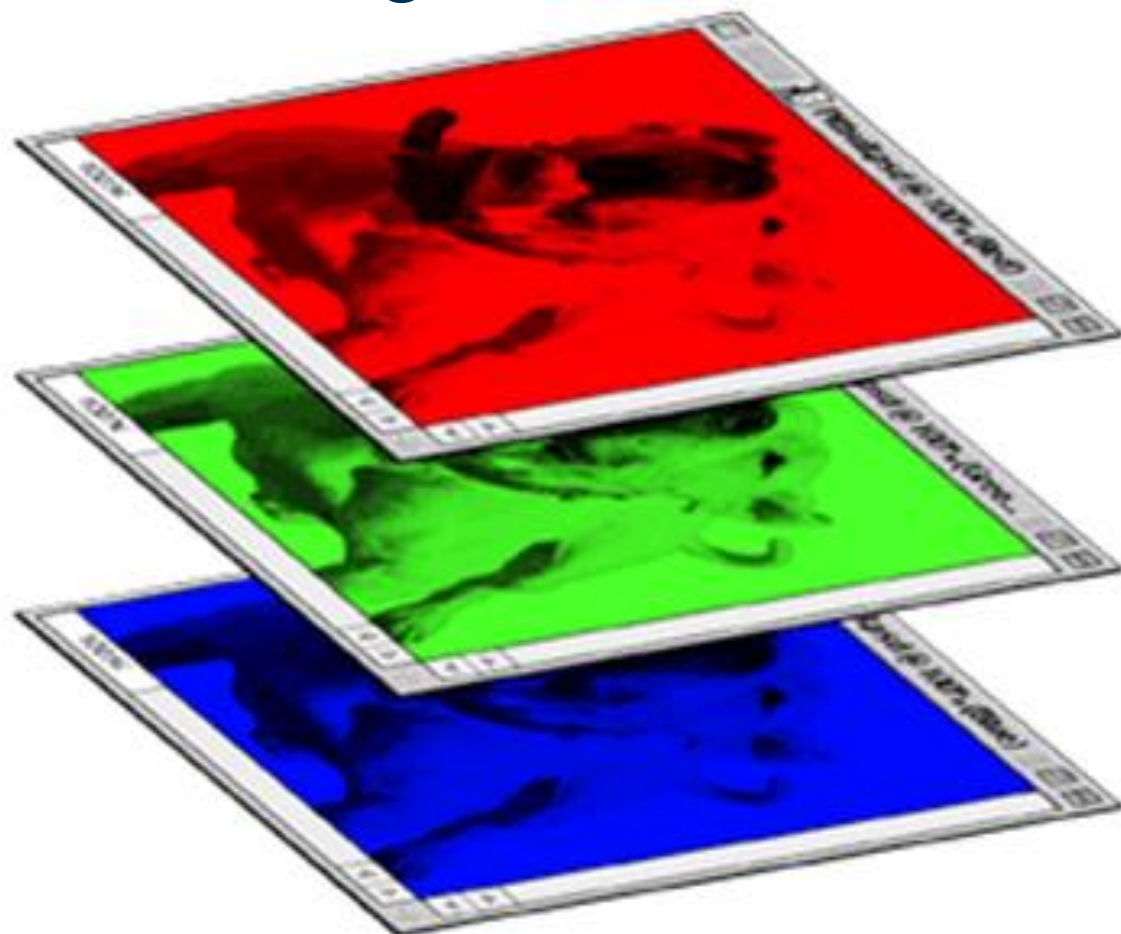
$$* \text{Sub}_{flat} = [\dots]_{1 \times 27}$$

$$\downarrow$$
 Scalar

3 layers if image has 3 layers

Filter 1

Color image



Filter 1

Filter 2

2x
4x4
feature
map

Quick Knowledge Check

- During backpropagation, why do we apply the chain rule? A. To move from input to output B. To compute derivatives layer by layer C. To skip intermediate layers D. To reset gradients
- If the gradient of a weight is positive, how should the weight change in gradient descent? A. Decrease the weight B. Increase the weight C. Keep it constant D. Randomly perturb it.
- What is one practical way to mitigate exploding gradients? A. Using smaller datasets B. Gradient clipping C. Removing activation functions D. Increasing batch size
- What does parameter sharing mean in CNNs? A. Each neuron has unique weights B. Different filters are random C. The same filter (set of weights) slides across all image locations D. Parameters are averaged
- What is the effect of using multiple filters in a convolutional layer? A. Network becomes shallower B. Filters average pixels C. It limits generalization. D. Each filter learns to detect a different visual pattern (edges, textures, shapes)
- Compared to a fully connected layer, a convolutional layer: A. Requires more parameters B. Uses local connections and shared weights C. Connects each neuron to every pixel D. Ignores spatial structure

Recap

- CNN excels at (for type of input data):

local feature analysis
grid type - images,

- CNN is efficient because:

Convolution → we don't have fully connected network

Weight Sharing

Multiple subproblems,
multiple filters to be learned

- Convolution:

filter $n \times n$ in 2D
subimage $n \times n$ in 2D) ← Dot Product.

Convolution

$n \times n$
image

filter kernel of size $k \times k$

Feature Map
 $n-k+1 \times n-k+1$

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

3×3

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

$m \times n \times n$

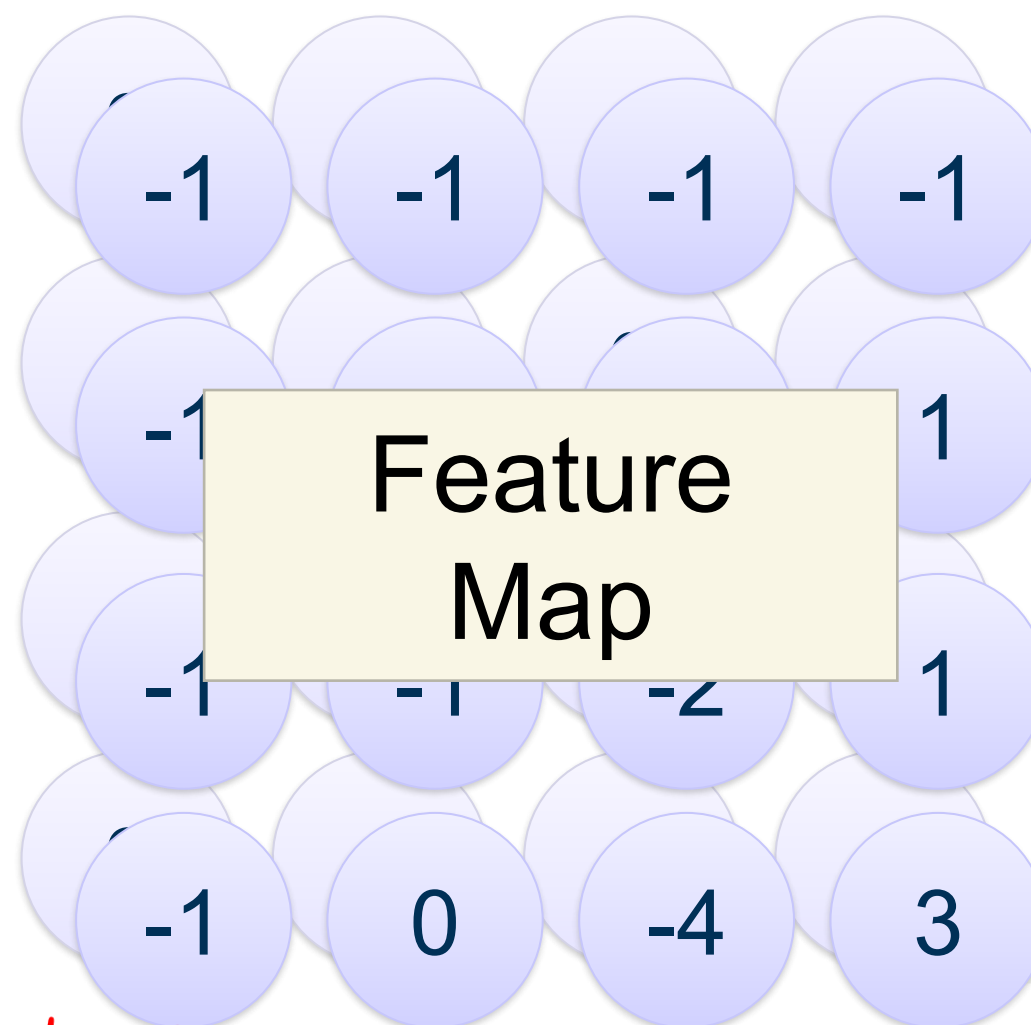
$m \times 3 \times 3$

Feature Map
 $(n-3+1 \times n-3+1)$

Two 4×4 images

Forming $2 \times 4 \times 4$ matrix

Repeat this for each filter



feature map

Convolution v.s. Fully Connected

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

image

1	-1	-1
-1	1	-1
-1	-1	1

-1	1	-1
-1	1	-1
-1	1	-1

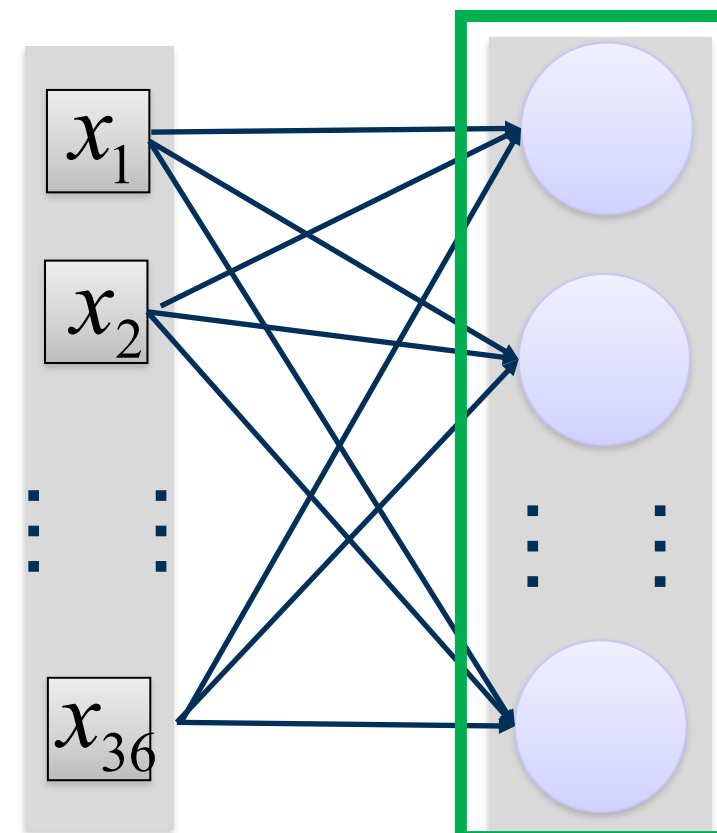


convolution

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Fully-
connected

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



$i=1$ 1	$i=2$ 0	$i=3$ 0	$i=4$ 0	$i=5$ 0	$i=6$ 1
$i=7$ 0	$i=8$ 1	$i=9$ 0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6x6

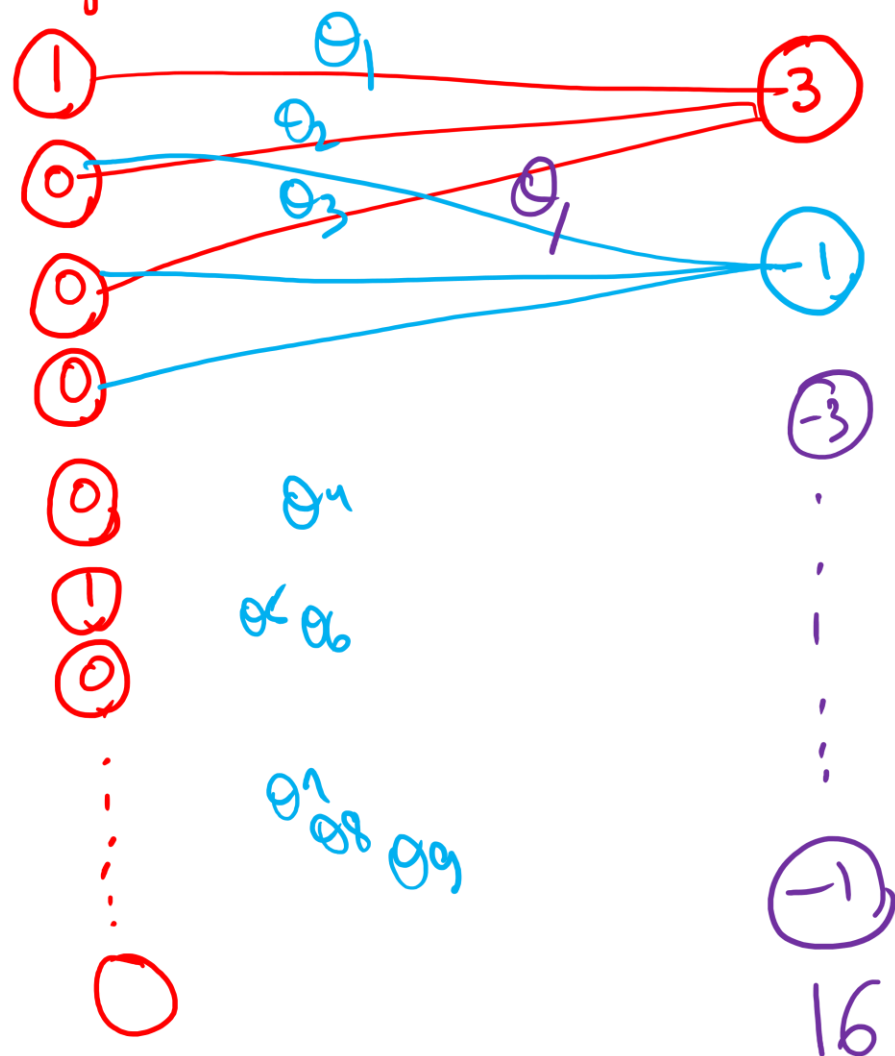
stride = 1

$\theta_1 = 1$	$\theta_2 = -1$	$\theta_3 = -1$
$\theta_4 = -1$	$\theta_5 = 1$	$\theta_6 = -1$
$\theta_7 = -1$	$\theta_8 = -1$	$\theta_9 = 1$

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Input image
neurons

1st layer
neurons



FCNN — 36x16

CNN — 9

Conventional Fully Connected layers (FC layers)

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

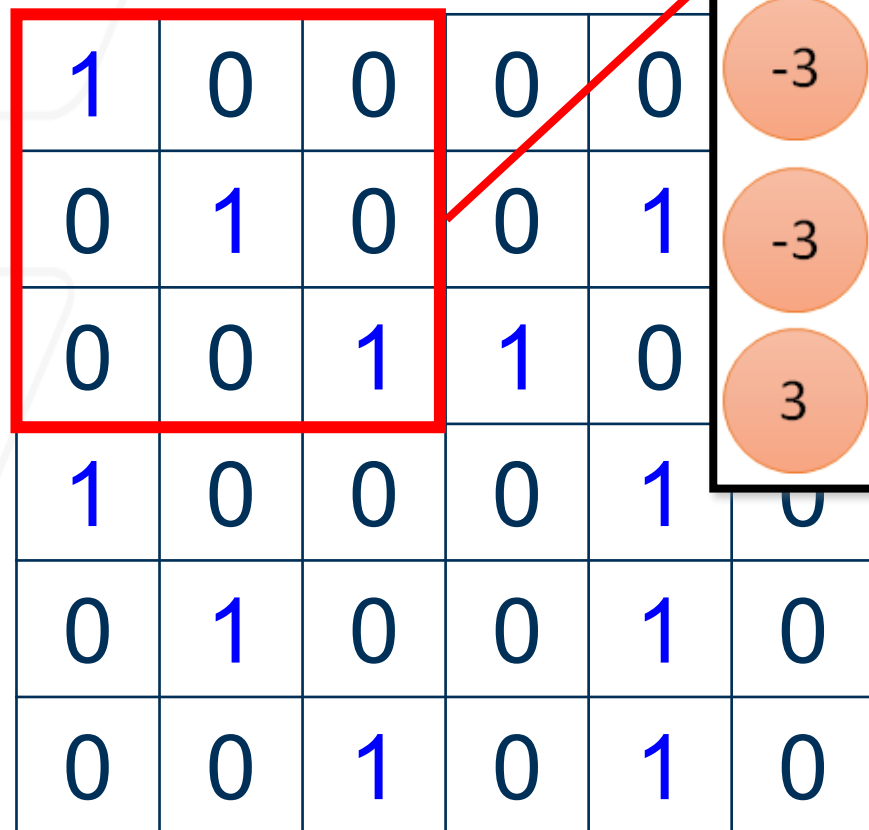
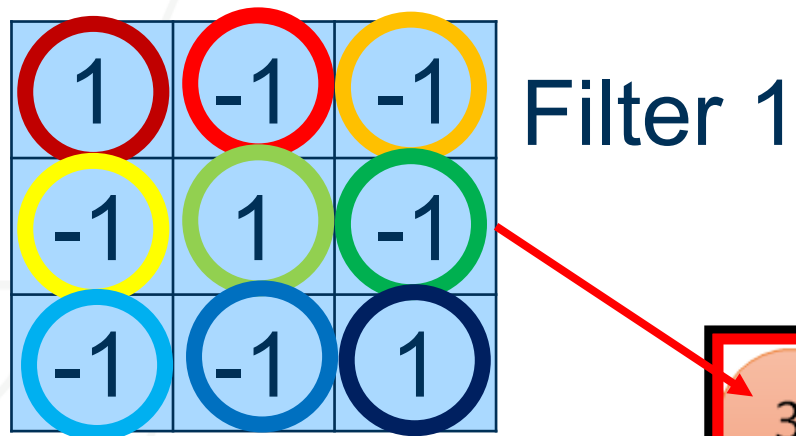
6 x 6 image

1	1	
2	0	
3	0	
4	0	
5	0	
6	1	
7	0	
8	1	
⋮		⋮
31	0	
32	0	
33	1	
34	0	
35	1	
36	0	

...

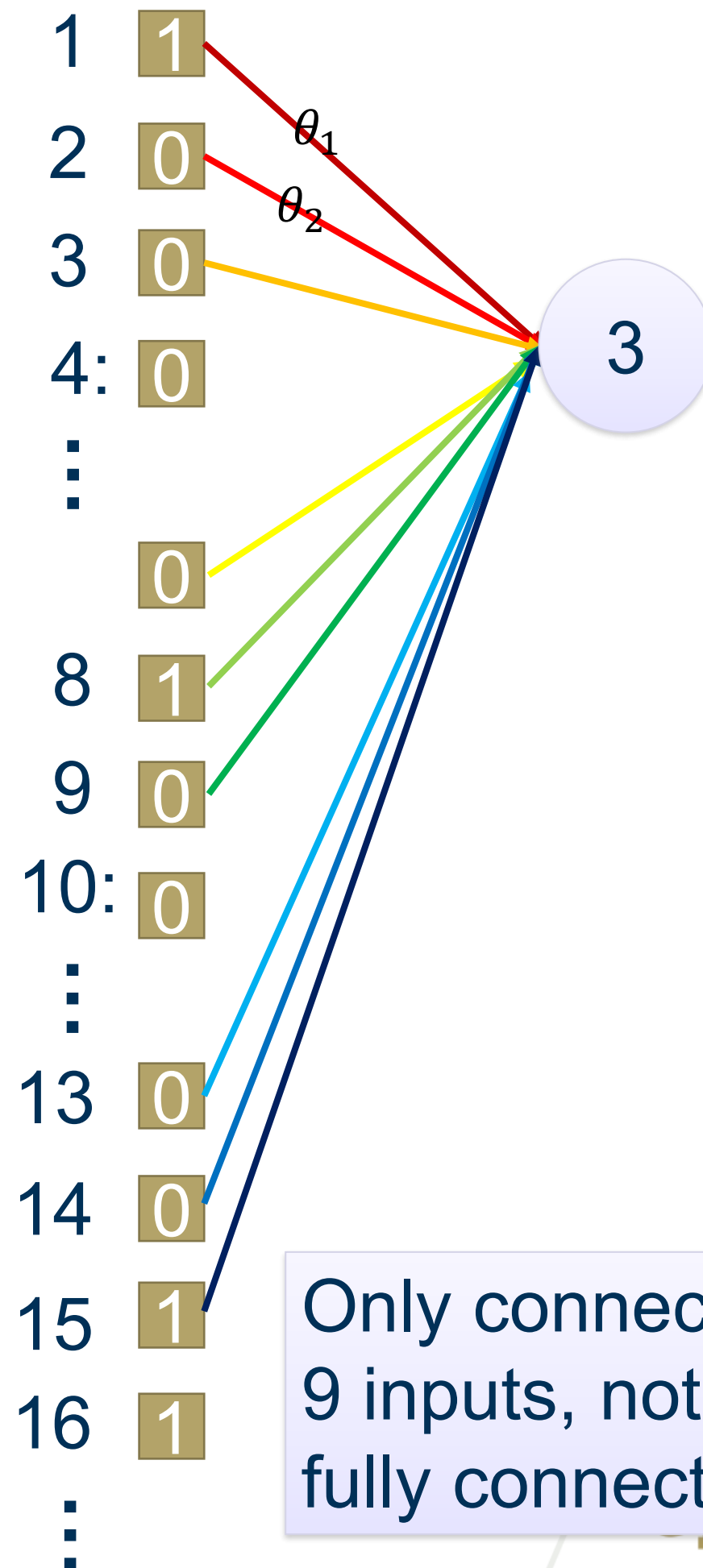
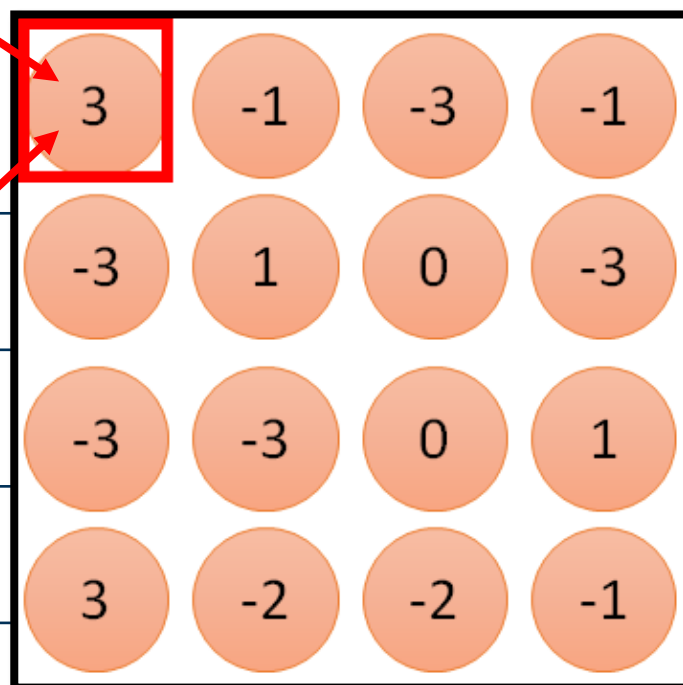
16

features 1st hidden layer

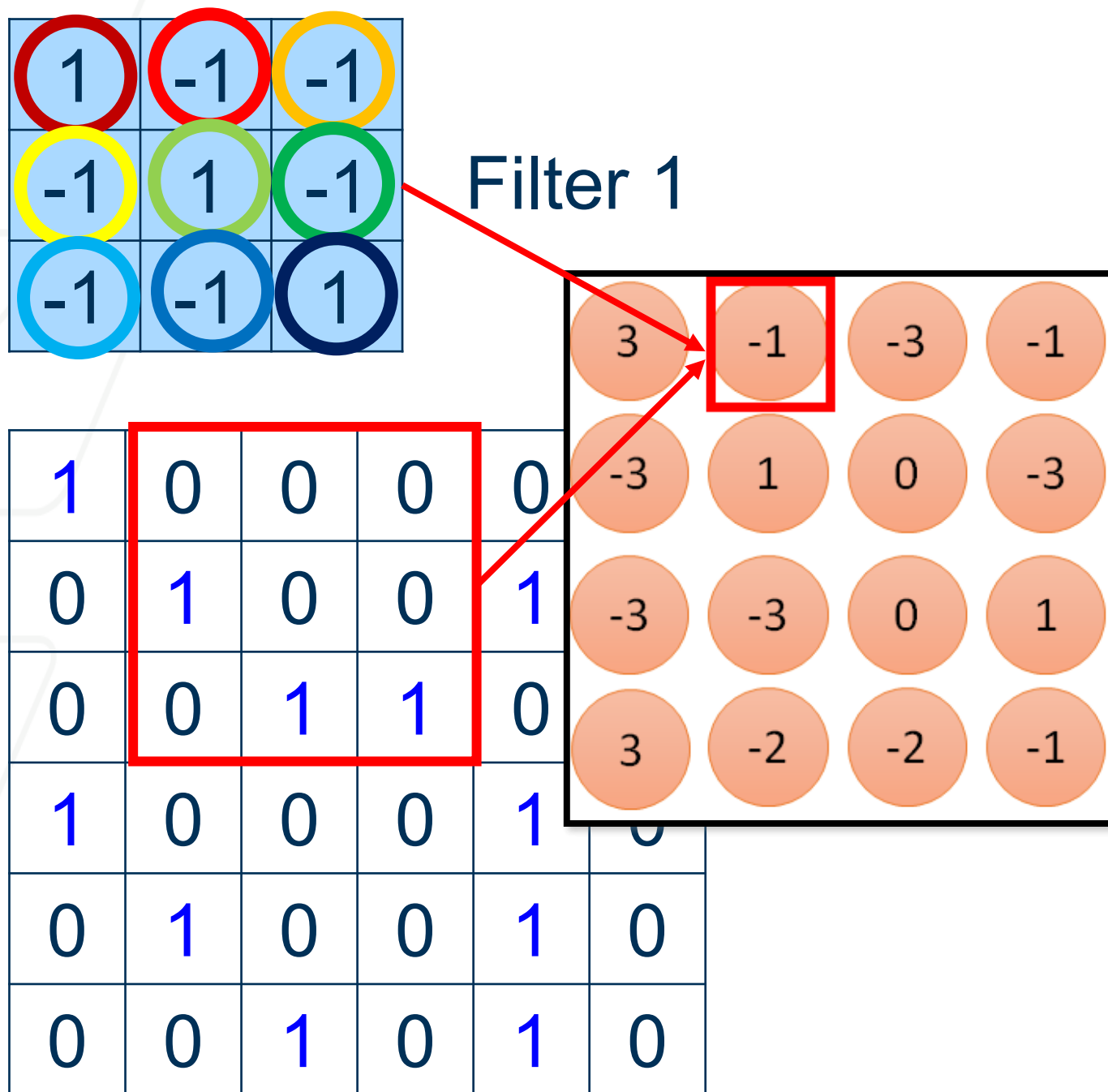


6 x 6 image

fewer parameters!



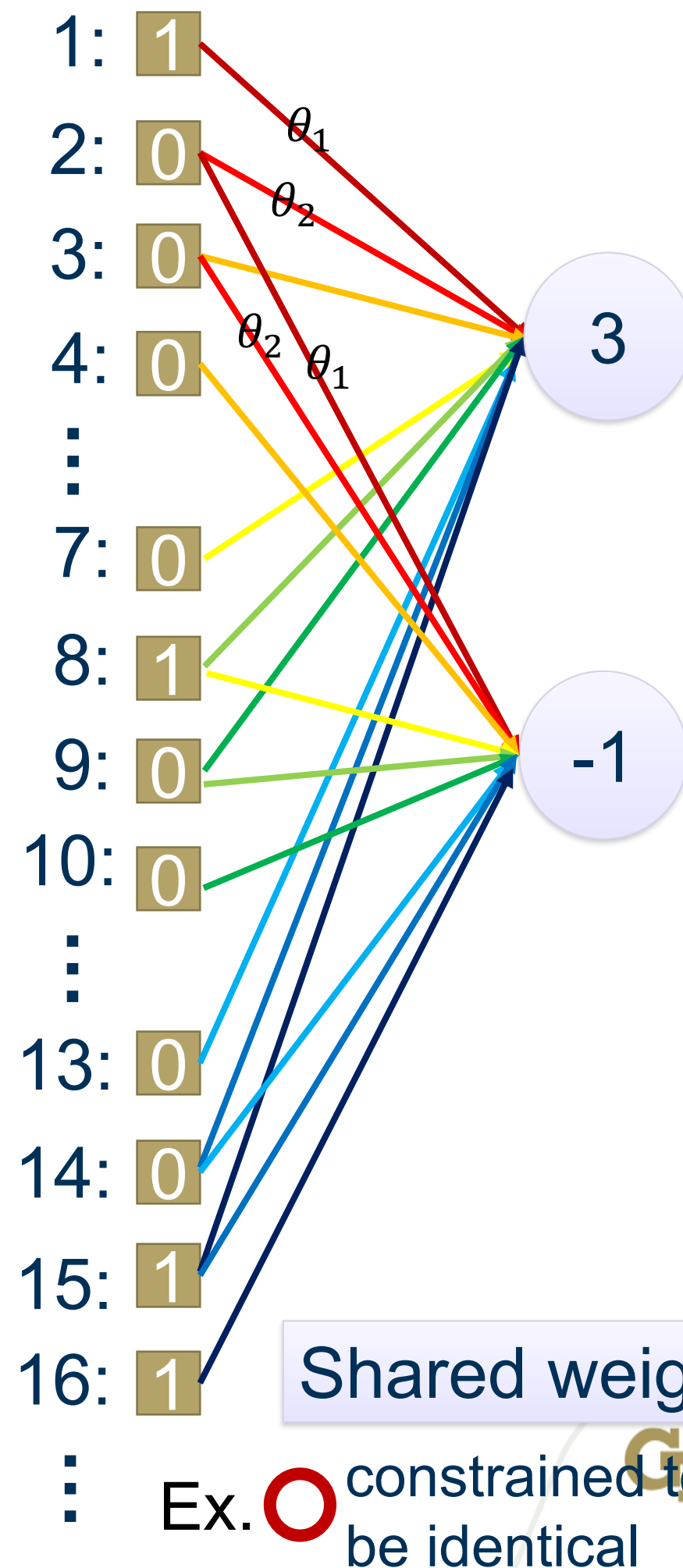
Only connect to 9 inputs, not fully connected



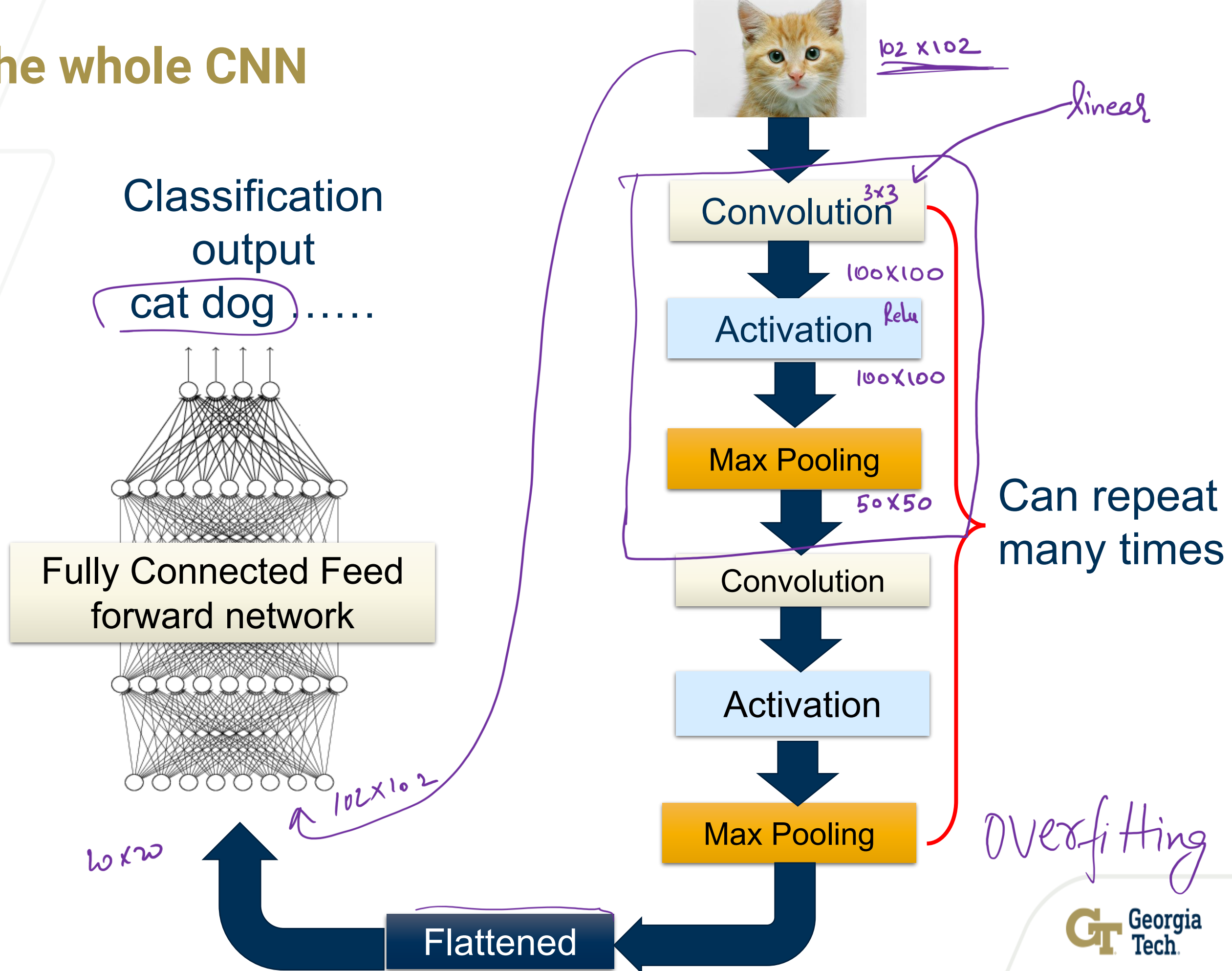
6 x 6 image

Fewer parameters

Even fewer parameters



The whole CNN



Max Pooling

6x6 input image

3x3 filter kernel

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

4x4 feature map

3	-1	-3	-1
-3	1	0	-3

-1	-1	-1	-1
-1	-1	-2	1

~~2x2 max pooling~~

-3	-3	0	1
3	-2	-2	-1

-1	-1	-2	1
-1	0	-4	3

Becomes 1/2

Why Pooling

- Subsampling pixels will not change the object

bird



Subsampling

bird



We can subsample the pixels to make image smaller

➡ fewer parameters to characterize the image

A CNN compresses a fully connected network in three ways. How are the number of parameters drastically reduced in CNN?

- Reducing number of connections due to convolution
- Shared weights on the edges
- Multiple features-multiple filters in same network
- Max pooling further reduces the complexity

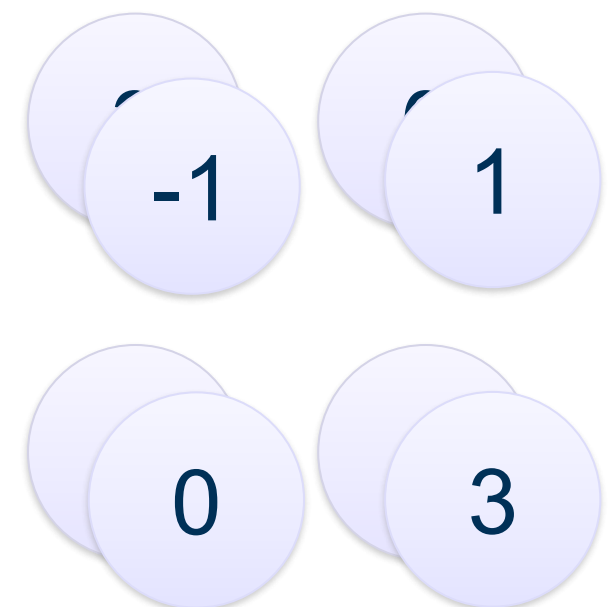
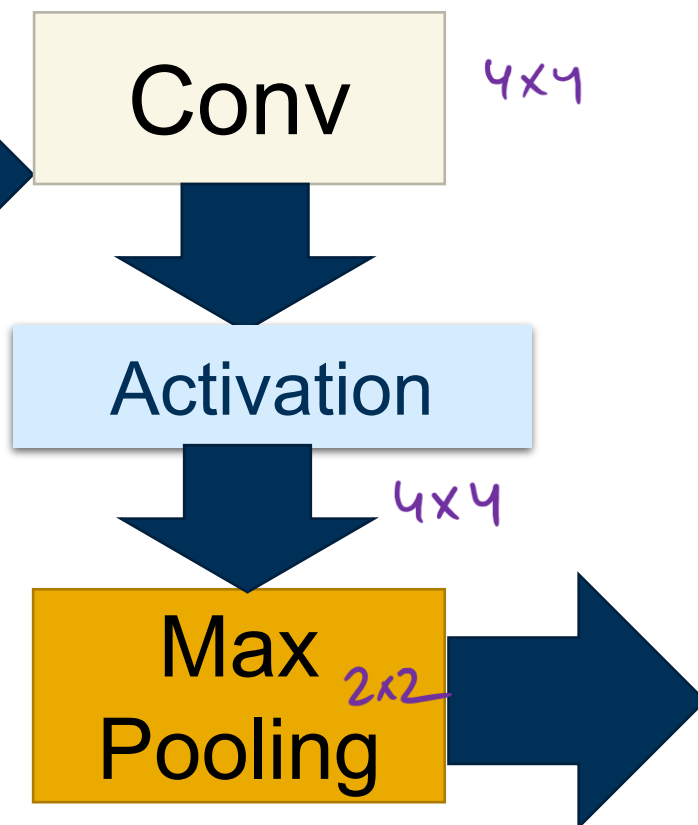
Max Pooling

New image
but smaller

Each filter
is a channel

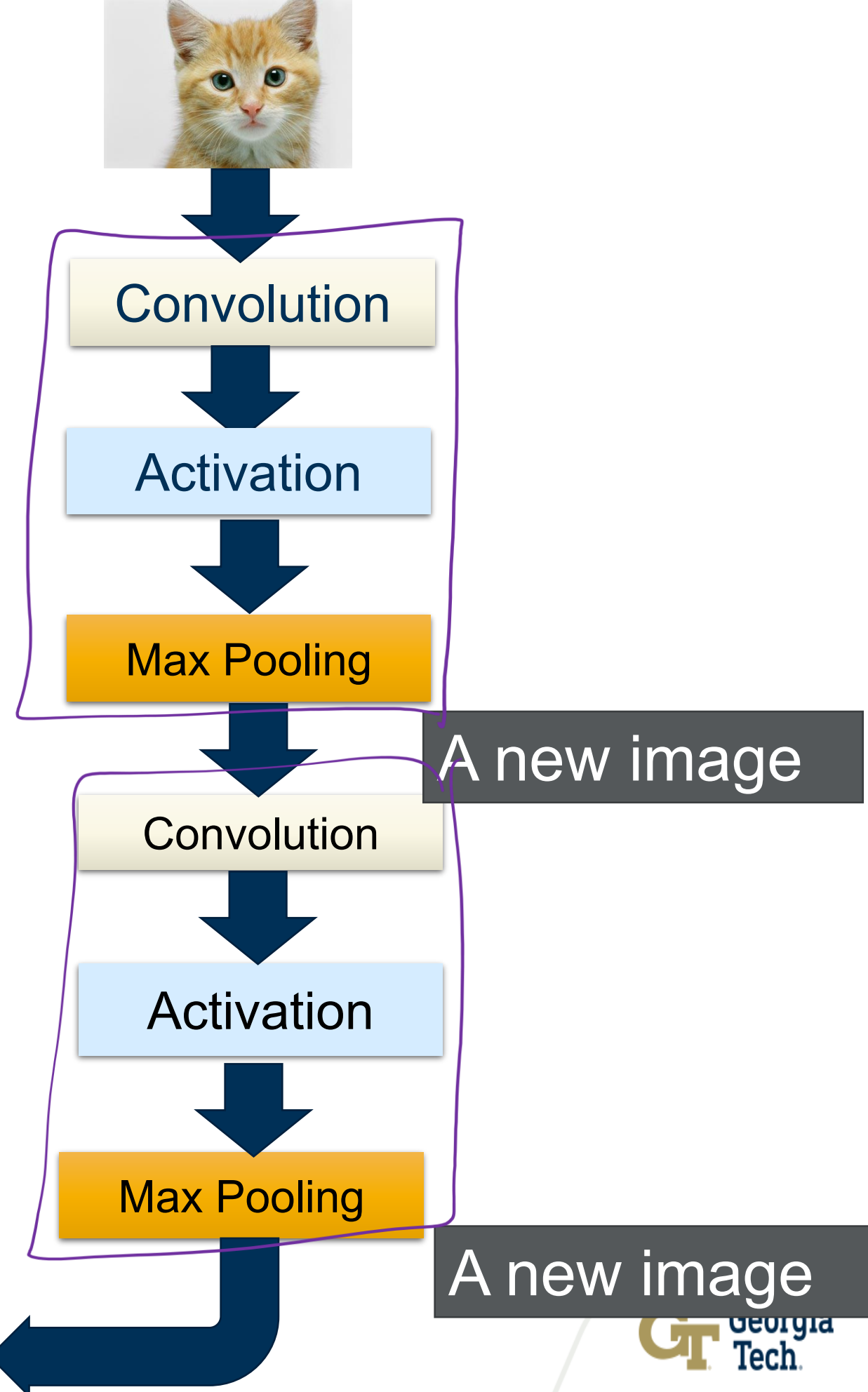
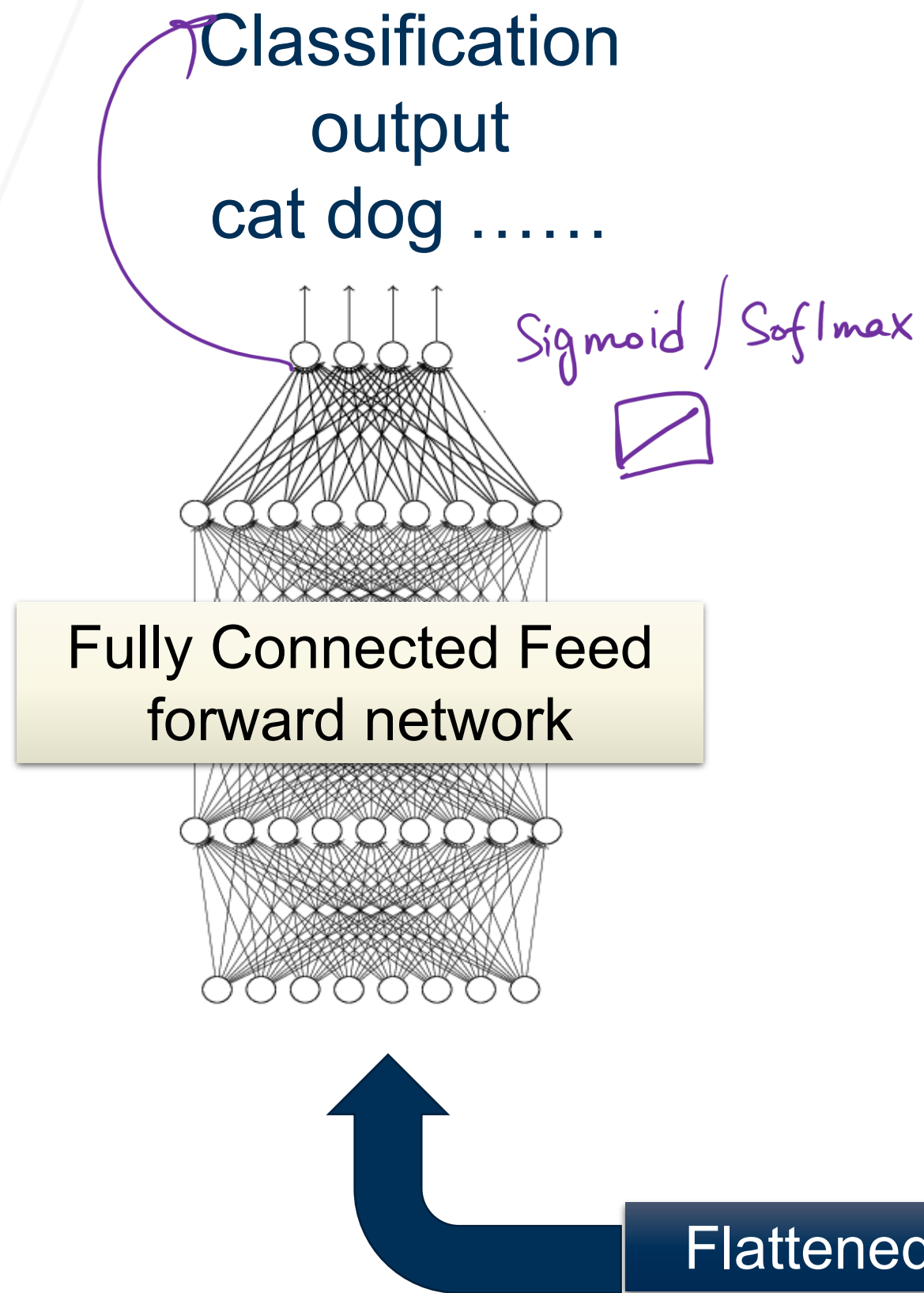
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

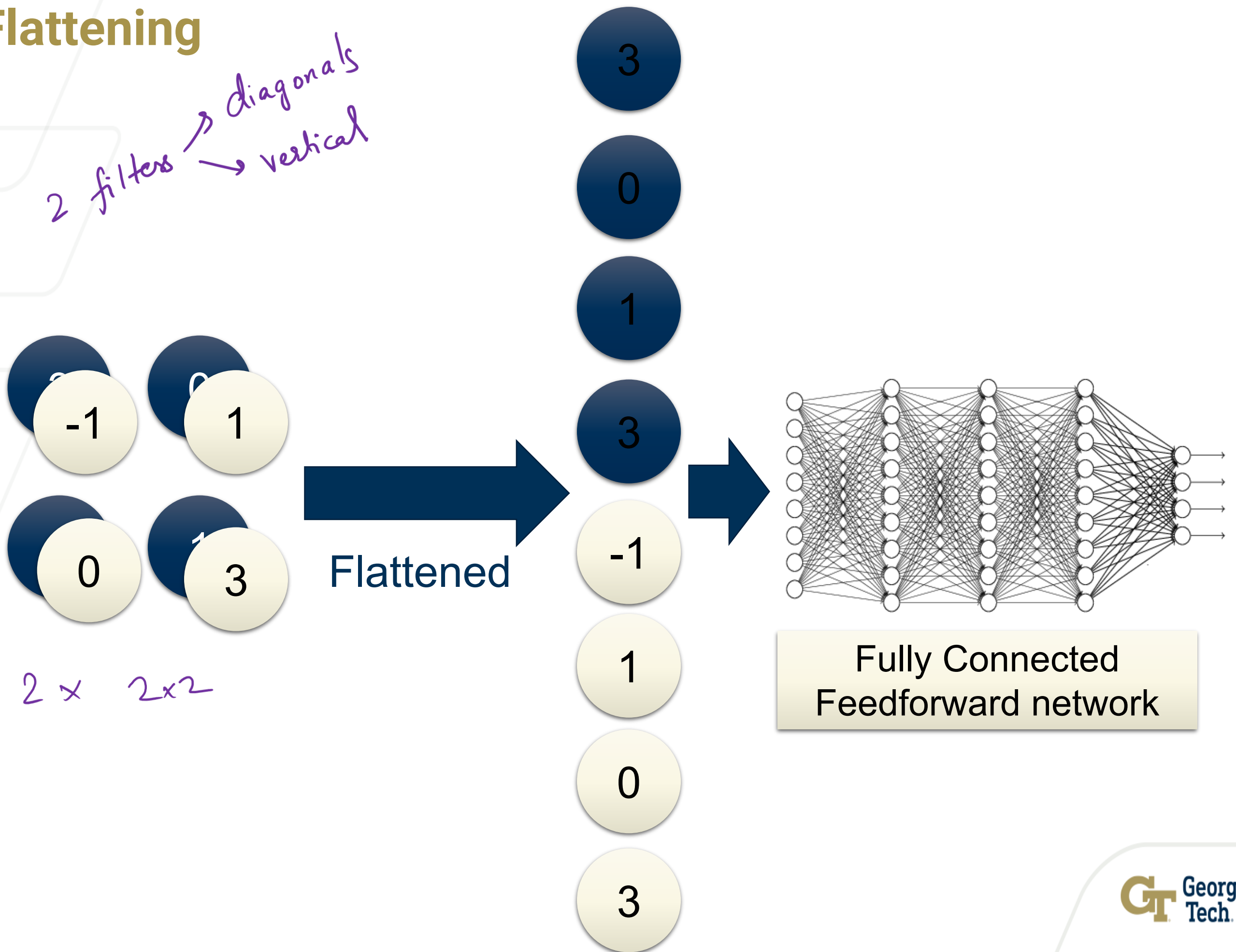


2 x 2 image

The whole CNN



Flattening



CNN in Keras

Only modified the *network structure* and *input format* (vector -> 3-D tensor)

initialized
`model2.add(Conv2D(25, (3, 3), input_shape=(28, 28, 1), activation='relu'))`
grayscale

1	-1	-1	1	-1	...
-1	1	-1	1	-1	...
-1	-1	-1	1	-1	...

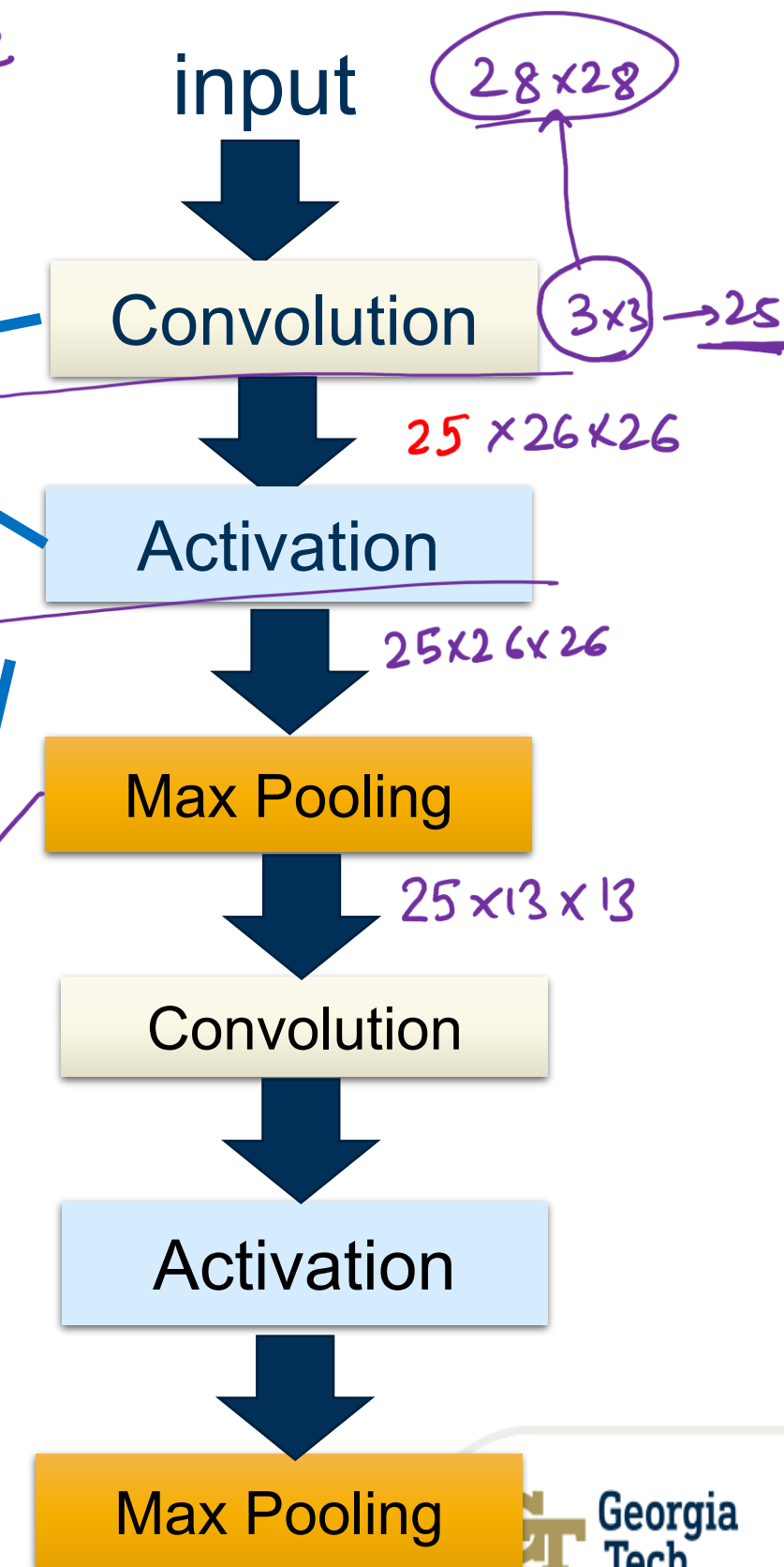
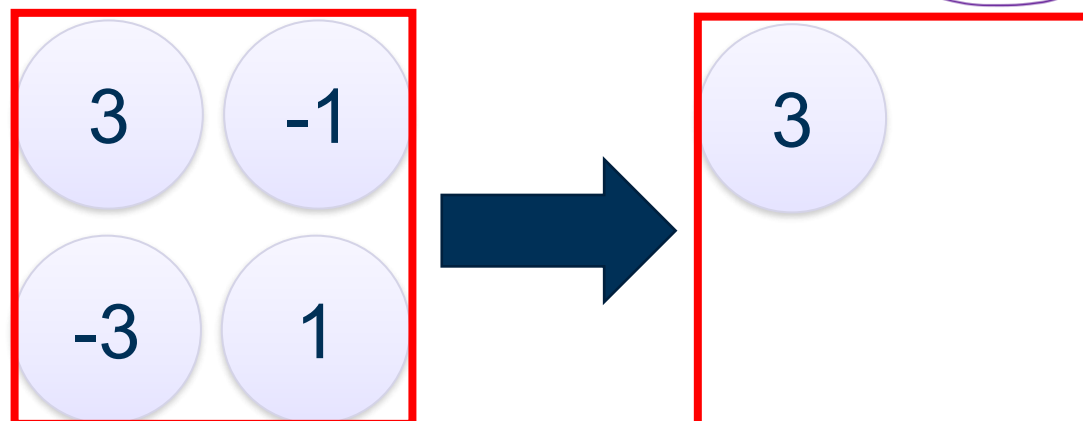
There are **25**
3x3 filters.

Input_shape = (28 , 28 , 1)

28 x 28 pixels

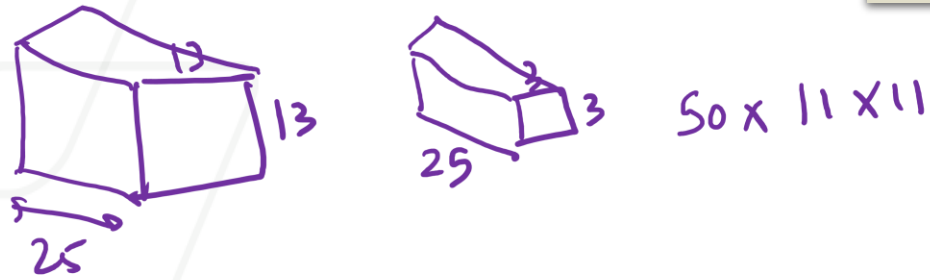
1: black/white, 3: RGB

`model2.add(MaxPooling2D((2, 2)))`



CNN in Keras

Only modified the *network structure* and *input format* (vector -> 3-D array)



1 x 28 x 28

```
model2.add( Convolution2D( 25, 3, 3,
                           input_shape=(28, 28, 1)) )
```

25 x 26 x 26

```
model2.add( MaxPooling2D( (2, 2) ) )
```

25 x 13 x 13

```
model2.add( Convolution2D( 50, 3, 3 ) )
```

50 x 11 x 11

```
model2.add( MaxPooling2D( (2, 2) ) )
```

50 x 5 x 5

Input

Convolution

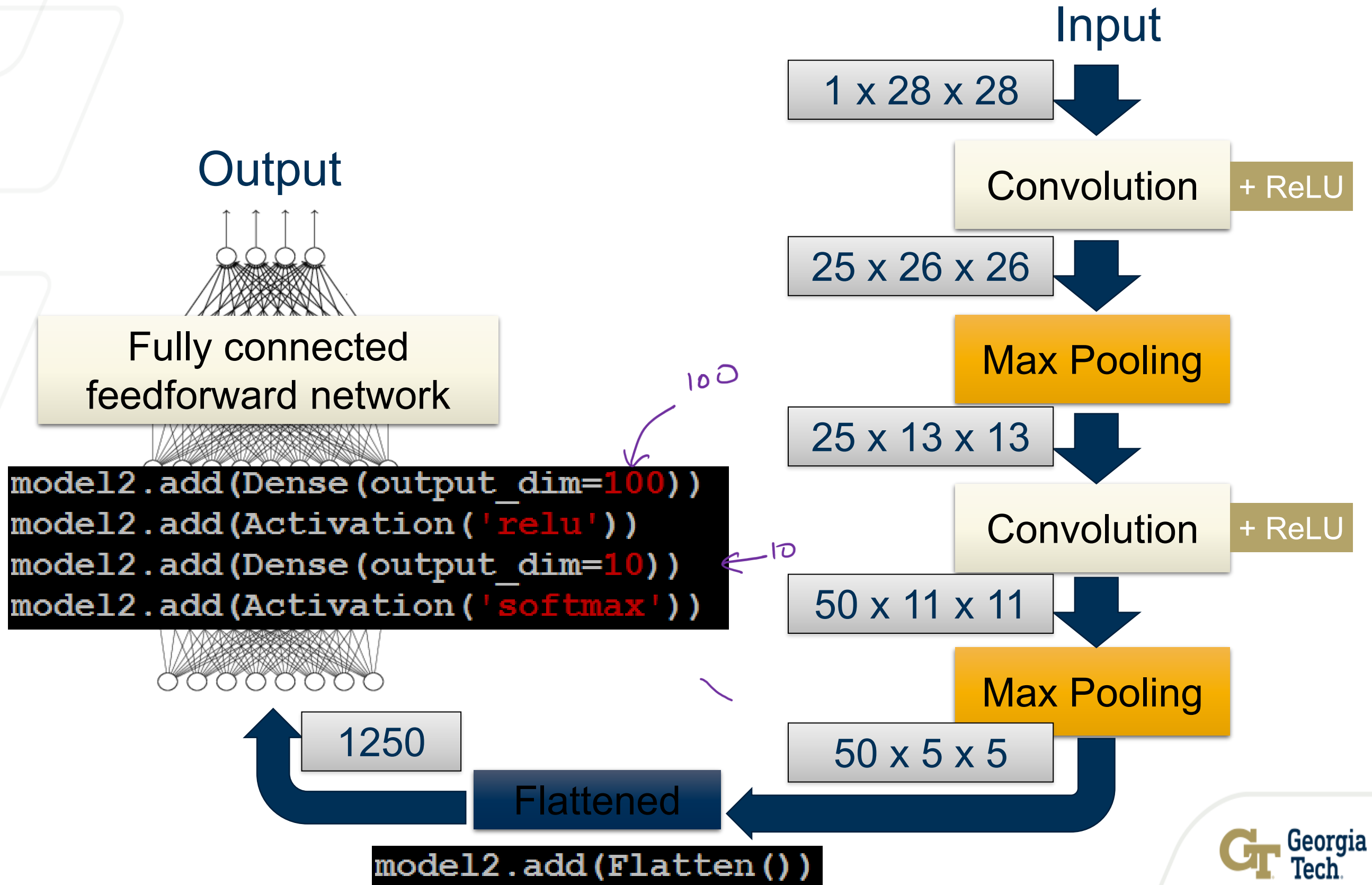
Max Pooling

Convolution

Max Pooling

CNN in Keras

Only modified the *network structure* and *input format* (vector -> 3-D array)

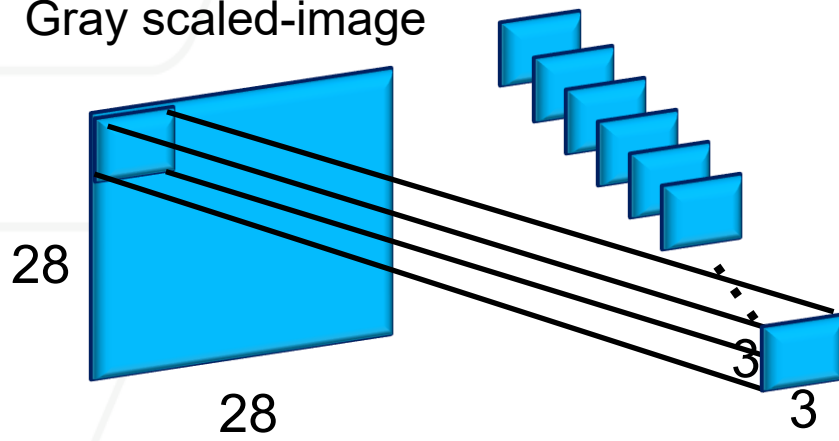


Number of Parameters

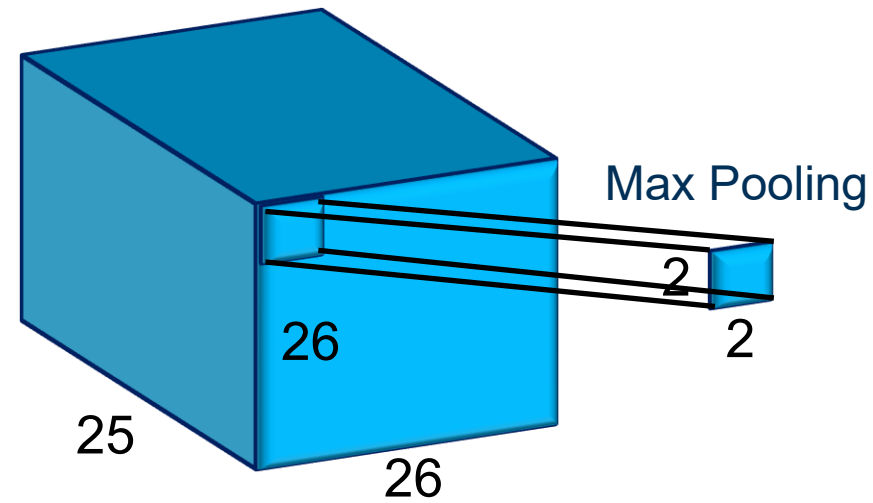
$25 \times 3 \times 3 + 25$ parameters
25 filters - Conv1
bias term

25: 3X3

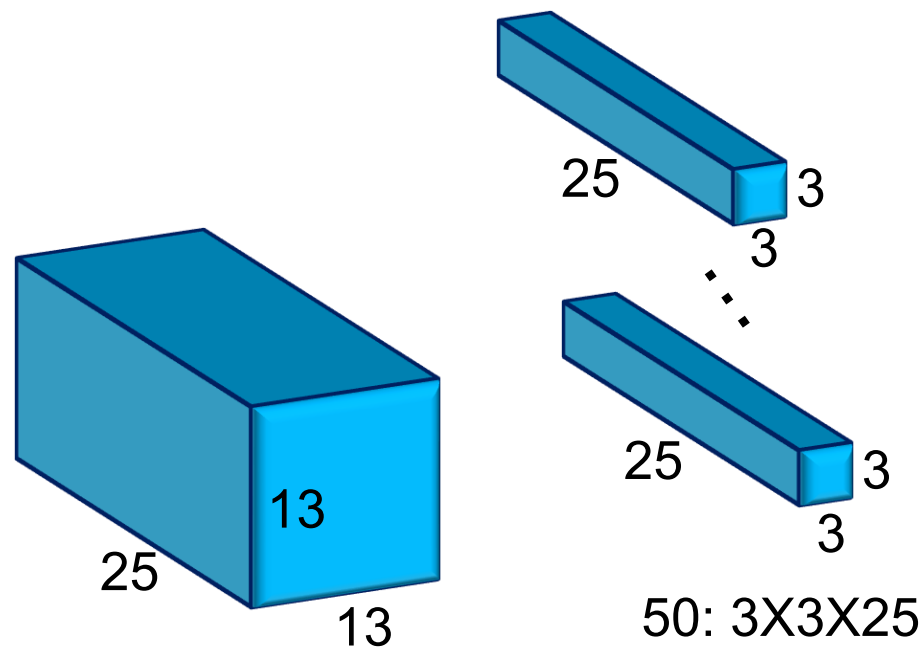
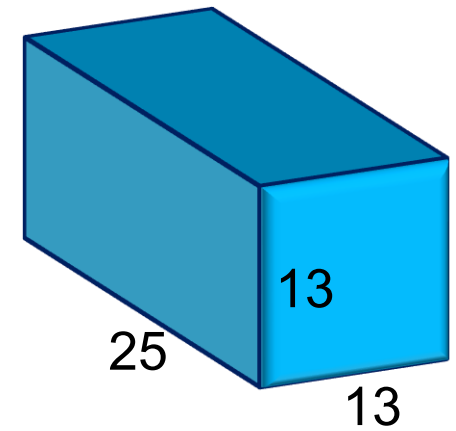
Gray scaled-image



Convolved result for 25 filters



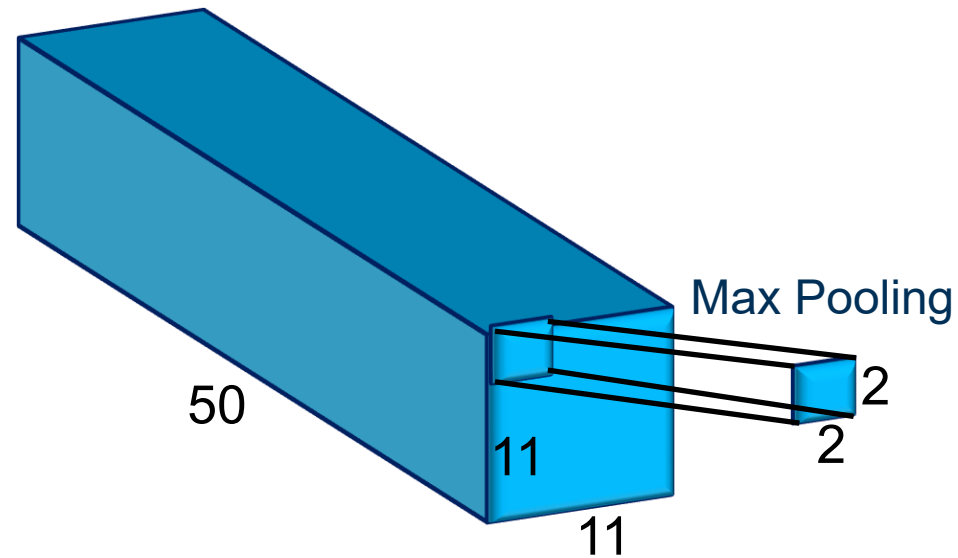
Result after Max Pooling



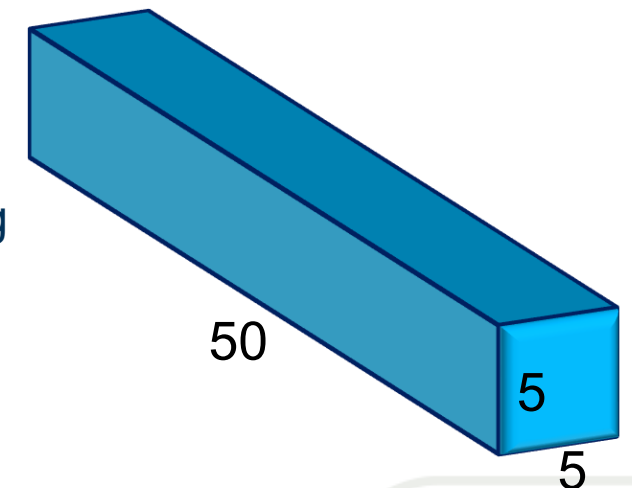
50 filters - Conv2

$50 \times 3 \times 3 \times 25 + 50$ parameters
bias

Convolved result for 50 filters

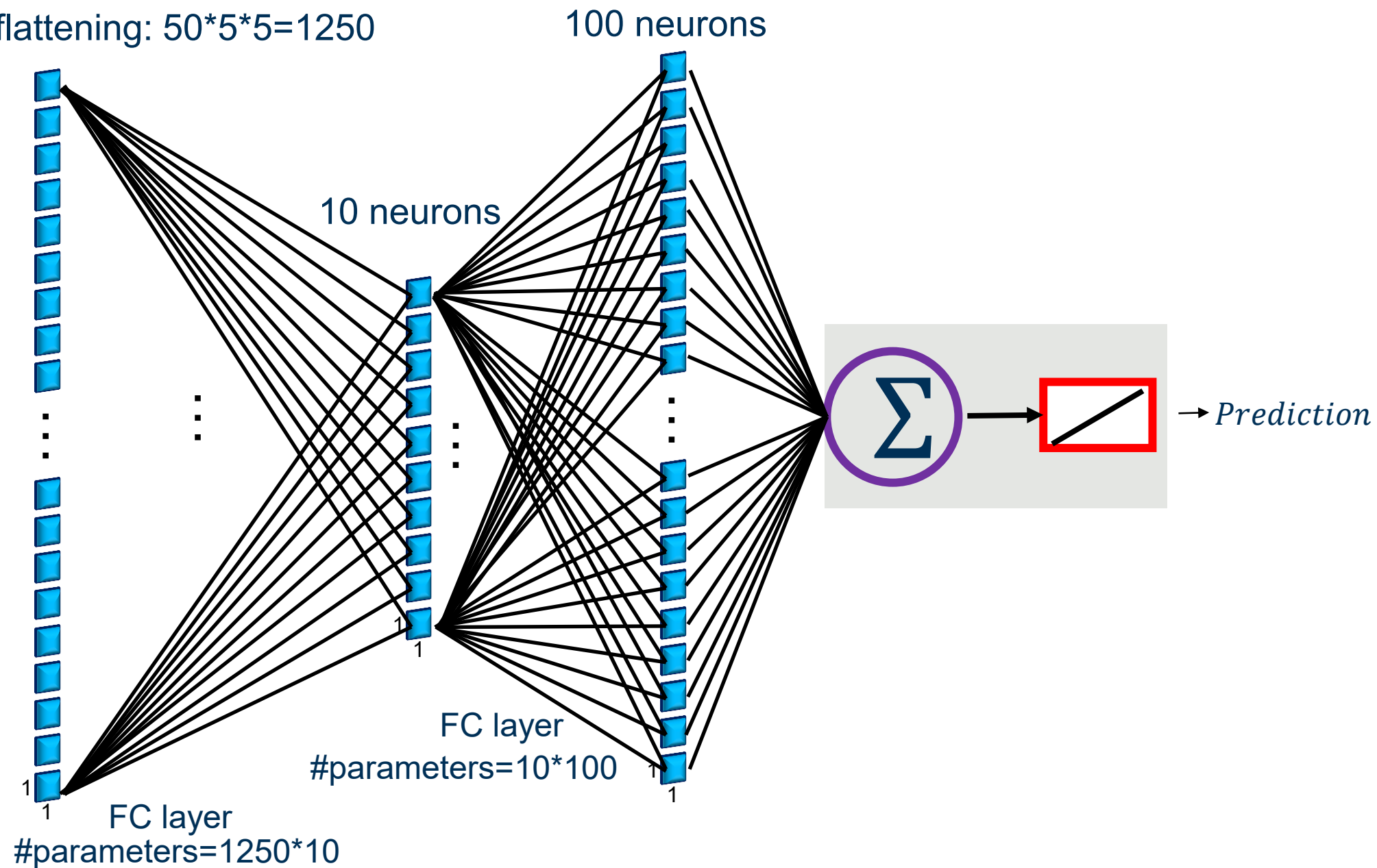
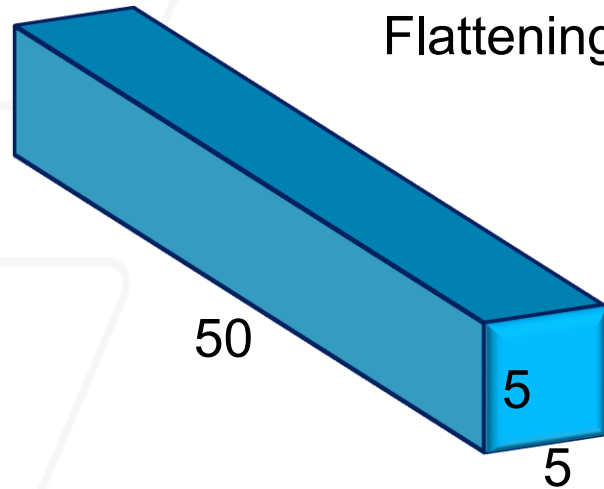


Result after Max Pooling



neurons after flattening: $50 \times 5 \times 5 = 1250$

Flattening



10 CNN Architecture

How CNNs Learn Filters

- CNN filters **start randomly**, not as edges or beaks.
- The **objective function** (loss) tells the model if its prediction is right or wrong.
- Backpropagation updates every filter so the final prediction improves.
- Over training, filters evolve into **meaningful pattern detectors** (edges, textures, beaks).

Role of Objective Function

- The **target label** defines the task (classification, presence/absence, localization).
- The **loss function** measures prediction error.
- Backprop uses this loss to adjust filter weights.
- Filters specialize in patterns that help reduce this error.

Beak Scenario 1: Presence/Absence (Binary Classification CNN)

- **Task: Does the image contain a beak?**
- **Objective Function:** Binary cross-entropy.
 - $\hat{y} = P(y=1)$
 - $BCE = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$
 - **Sigmoid** at the output (because we need a single probability).
- **Architecture:** Standard CNN
- **Learned Filter:**
 - A filter becomes a **generic beak detector** because it helps predict the label.
- Only image-level yes/no prediction.

Beak Scenario 2: Internal Feature Detector (Classification CNN)

- **Task: Classify whole image** (e.g., “sparrow vs dove vs hawk”).
- **Objective Function:** Cross-entropy classification loss.
 - You penalize the model if the **probability for the correct class** is low.
 - **Softmax** at the output (because we need a full probability distribution).
- **What Filters Learn:**
 - Early layers → edges, curves, textures
 - Mid layers → **beak detector** emerges because it helps differentiate species
 - **No output says “beak present.”**

Beak Scenario 3: Localization (Object Detection)

- **Task: Find the beak's location + optional class.**
- **Objective Function:**
 - Classification loss (is it a beak?)
 - Localization loss (Smooth L1 loss) for bounding box
- **Architecture:**
 - Faster R-CNN, YOLO, SSD, RetinaNet
- **Filters Learned:**
 - Low-level: edges
 - Mid-level: **beak filters**
 - Detection head maps activations → bounding box coordinates